VIP – AI Hub for Config-Driven Agent Lifecycle Management, Orchestration, and Observability

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Ibtihel Mnaja

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : Ahmed Rebai

Cachet & Signature / Stamp & Signature

Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : Jihen Jebri

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné**/This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover page.

ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn
Siége Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889
Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454

esprit

Se former autrement

2025 - 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : Data Science

VIP — Agentic AI Hub for Config-Driven Agent Lifecycle Management and Orchestration

By: *Ibtihel Mnaja*

Academic supervisor: *Jihen Jebri*

Corporate Internship Supervisor: *Ahmed Rebai*

VALUE

Abstract

Agentic Artificial Intelligence refers to AI systems capable of performing tasks through autonomous or semi-autonomous agents that can reason, use tools, interact with external systems, and collaborate within structured workflows. In the financial services industry, the adoption of such systems requires not only advanced AI capabilities, but also strong guarantees in terms of scalability, control, auditability, and operational reliability.

This internship report presents the design and implementation of an **AI Agentic Hub Platform** aimed at building, deploying, and operating intelligent applications at scale. The proposed platform is based on a modular multi-agent architecture in which agents are generated from declarative configurations, deployed as independent containerized services, and coordinated through an orchestration layer. This approach makes it possible to create specialized agents without modifying their source code, by relying on configurable prompts, tools, model parameters, and input/output schemas.

The work focuses on several key aspects of enterprise AI deployment, including agent generation, workflow orchestration, schema-based communication, prompt and schema versioning using MLflow, Docker-based containerization, and observability. The objective is to provide a controlled and extensible blueprint for developing financial AI applications while reducing fragmentation, improving governance, and ensuring traceability across the full lifecycle of intelligent systems.

Keywords: Agentic AI, Multi-Agent Systems, AI Orchestration, Docker, MLflow, Financial Services, Schema Validation, Observability.

Summary

This internship project focuses on the design and implementation of **VIP – Value Intelligent Platform**, an **Enterprise-Grade Agentic AI Hub** aimed at building, deploying, and operating intelligent applications at scale.

- **Dynamic Agent Generation:** Developed a configurable Agent Factory capable of creating specialized AI agents from declarative configurations, without modifying the source code.
- **Modular and Containerized Architecture:** Designed agents as independent FastAPI services deployed in isolated Docker containers, ensuring scalability, separation of concerns, and easier maintenance.
- **Centralized Governance and Versioning:** Integrated MLflow to manage prompts, schemas, and configuration artifacts, enabling traceability, version control, and a "Zero Code Change" approach.
- **Intelligent Orchestration:** Implemented an orchestration layer supported by an adapter to coordinate multi-agent workflows and manage schema transformations between agents.

Conclusion: The platform provides a scalable and reusable blueprint for enterprise AI applications, allowing organizations to transform complex business processes into modular, auditable, and intelligent agent-based workflows.

Acknowledgments

I would first like to express my sincere gratitude to the entire team at **Value Digital Services** for welcoming me into such a professional, collaborative, and innovative environment. The experience gained throughout this internship has been both enriching and inspiring, and I truly appreciated the support and positive atmosphere within the company.

I am deeply grateful to my professional supervisor, Mr. **Ahmed REBAI**, for his guidance, technical expertise, and continuous support throughout this project. His vision, advice, and constructive feedback played a major role in the successful realization of the *VIP – Value Intelligent Platform*.

I would also like to sincerely thank my academic supervisor, Ms. **Jihen JERBI**, for her valuable supervision, encouragement, and academic guidance. Her support helped ensure the quality and rigor of this work throughout the internship period.

On a personal note, I would like to express my deepest gratitude to my parents, **Hayet** and **Ali**, for their unconditional love, sacrifices, and constant encouragement throughout my studies. Their trust and support have always been my greatest source of motivation.

I would also like to warmly thank my sister **Dhouha**, as well as my brothers **Borhen** and **Louey**, for their continuous encouragement, patience, and support during both the challenging and rewarding moments of this journey.

Finally, I would like to thank everyone who contributed, directly or indirectly, to the success of this internship experience and to the completion of this project. This experience has been a major step in both my academic and professional journey, and I am truly grateful for the knowledge and experience I have gained.

Table of Contents

List of Figures

List of Tables

Glossary

ADLC:	Agent Development Lifecycle
AI:	Artificial Intelligence
API:	Application Programming Interface
IBM:	International Business Machines
LLM:	Large Language Model
QA:	Quality Assurance
REST:	Representational State Transfer
UI:	User Interface
UX:	User Experience
VIP:	Value Intelligent Platform

Introduction

The recent evolution of Artificial Intelligence has marked a major shift from traditional conversational models toward *agentic systems*. These systems are characterized by their ability to reason, use external tools, interact with data sources, and execute complex multi-step workflows with a certain level of autonomy. In this context, the subject of this internship focuses on the design and implementation of **VIP – Value Intelligent Platform**, an enterprise-grade Agentic AI Hub intended to build, deploy, and operate intelligent applications at scale across different business domains.

Despite the significant progress of Large Language Models, their integration into enterprise environments remains challenging. The main difficulty lies in transforming AI prototypes into reliable, reusable, and governable systems that can be deployed in real operational contexts. Traditional approaches often rely on hard-coded agents, making them difficult to adapt, maintain, or extend. The problem addressed in this project is therefore to design a modular platform capable of generating specialized agents from declarative configurations, while ensuring scalability, traceability, and operational control.

The adopted approach is based on a decoupled and containerized multi-agent architecture. Each agent is designed as an independent FastAPI service, deployed in a Docker container, and configured through prompts, tools, model parameters, and input/output schemas. MLflow is used to manage and version configuration artifacts such as prompts and schemas, while an orchestration layer coordinates the execution of agent workflows. A semantic adapter is also introduced to manage schema transformations between agents and ensure smooth communication across the platform.

This report is organized into five chapters detailing the progression of the work carried out during the internship. **Chapter 1** presents the general context of the internship and the host company. **Chapter 2** introduces the preliminary work completed during the onboarding phase. **Chapter 3** presents the overall architecture of the VIP platform. **Chapter 4** details the design and implementation of the agent generation mechanism and the core platform components. **Chapter 5** focuses on orchestration, observability, and evaluation aspects. Finally, a **conclusion** summarizes the main contributions of the project and proposes possible future improvements.

Chapter 1

Introduction and Project Context

1.1 Introduction

The rapid adoption of AI agents and generative language models, including large language models (LLMs) and small language models (SLMs), is creating new opportunities for enterprise automation while increasing the need for reliable orchestration, deployment, and observability. In this context, my internship at **Value Digital Services** focused on building a modular, config-driven platform for agent generation and management, alongside the evaluation of small language models through SLM-evals. This chapter introduces the host company, clarifies the project goals and problem context, and presents the methodology followed throughout the internship.

1.2 Host Company and Areas of Expertise

Value Digital Services is a Tunisian company founded in 2019 and specialized in digital transformation and AI consulting. Its activities combine software engineering, data engineering, artificial intelligence, and strategic consulting, which made it an appropriate environment for an internship focused on building an enterprise AI agent platform and evaluating small language models.

Figure 1: Logo of Value Digital Services

- **Digital Factory:** software engineers, DevOps engineers, QA engineers, business analysts, and UX/UI designers build and deliver modern digital products.
- **Data Factory:** data engineers and data analysts design data pipelines, analytics workflows, and data platforms that support decision-making.

- **AI Factory:** AI researchers and machine learning engineers design and deploy AI-driven solutions, including generative AI and intelligent automation systems.
- **Advisory:** consultants support clients in structuring digital transformation initiatives, aligning technical choices with business and governance needs.

These poles work together to deliver modern digital solutions, from consulting and design to data infrastructure, AI implementation, deployment, and operational support.

1.3 Project Overview

The project, entitled **VIP – Value Intelligent Platform**, aims to design and implement an enterprise-grade AI agent orchestration platform capable of building, deploying, and managing intelligent applications through a modular multi-agent architecture. Unlike traditional monolithic AI systems, the platform follows an *Agent-as-a-Service* approach, where specialized agents are dynamically generated from declarative configurations and deployed as independent containerized services.

VIP provides a unified environment for agent creation, workflow orchestration, observability, and lifecycle management. Users can configure agents by selecting models, prompts, schemas, providers, and tools without directly modifying the source code. The generated agents can then be integrated into multi-agent execution pipelines coordinated through an orchestration layer and monitored through built-in observability services.

This platform directly addresses the gaps identified in existing solutions by combining support for multiple LLM providers and open-source models with a modular architecture, declarative configuration management, and built-in observability. This design reduces dependency on a single proprietary provider, simplifies agent creation and deployment, and makes agent behavior easier to trace, evaluate, and improve over time.

Alongside VIP, I developed the **SLM-evals** side project, a reproducible evaluation pipeline for benchmarking LLM and SLM providers. It compares OpenAI, Groq, Phi-4, Llama 3.1, and other alternatives under controlled prompts and golden datasets, while tracking latency, token usage, and layered evaluation metrics. The results of this pipeline guide the choice of models for the *LLM-provider* layer and support VIP's objective of maintaining provider flexibility while selecting models that fit performance, cost, and reliability requirements.

1.4 Problem Statement

Despite the rapid evolution of AI agents and large language models, the deployment of agentic systems in enterprise environments remains complex. These challenges matter because enterprises need systems that can scale reliably, remain auditable, control operational costs, and comply with internal governance and security requirements. When agent platforms lack provider flexibility, observability, automation, or modular orchestration, they become difficult to deploy at scale and hard to maintain in production.

The main challenges addressed by this project can be summarized as follows:

- **High Operational Costs and Vendor Lock-in:** Existing AI agent solutions can be expensive to operate at scale, especially when they depend heavily on proprietary LLM APIs. This dependence increases costs, limits provider flexibility, and makes it harder to switch between OpenAI, Groq, Azure, open-source LLMs, or SLM alternatives.
- **Poor Observability:** Many current systems lack sufficient traceability, monitoring, and execution tracking. This limits debugging, performance evaluation, and the audit evidence required for compliance and governance.
- **Absence of Automated Deployment:** Creating and deploying new agents is often a manual and time-consuming process. Without automated deployment mechanisms, it becomes difficult to rapidly generate, configure, and launch agents in a consistent way.
- **Lack of Orchestration and Modularity:** Existing approaches often do not provide strong workflow orchestration capabilities. Agents are not always designed as reusable and modular components, which limits workflow composition and makes multi-agent collaboration harder to manage.

Based on these limitations, the central problem of this project can be formulated as follows:

> How can we design a modular, secure, observable, and cost-optimized AI agent orchestration platform that enables dynamic agent creation, automated deployment, and workflow composition while evaluating and integrating multiple LLM and SLM providers to reduce vendor lock-in, control costs, and maintain strong performance?

.

1.5 Objectives

Based on the problem statement, the objectives of the project are divided into functional and non-functional objectives.

Functional Objectives

- Enable dynamic agent creation from declarative YAML or JSON configuration files.
- Provide a visual workflow builder for composing and managing multi-agent execution pipelines.
- Offer a unified interface for selecting models, prompts, schemas, providers, and tools without modifying the source code.
- Support reusable multi-agent pipelines that can combine specialized agents into structured workflows.
- Integrate SLM-evals results into the provider selection process to compare LLM and SLM options before deployment.

Non-functional Objectives

- Reduce vendor lock-in and operational cost by evaluating multiple providers, including proprietary LLM APIs and local SLM alternatives.
- Optimize deployment costs through containerized, configurable, and reusable agent services.
- Ensure observability and traceability by monitoring execution traces, token usage, latency, and estimated cost.
- Preserve modularity and scalability by allowing agents, tools, providers, and workflows to evolve independently.
- Strengthen security and provider independence by limiting hard-coded provider dependencies and supporting controlled configuration management.

1.6 Technology Choices

The project relies on a modular technology stack selected to support agent generation, orchestration, deployment, observability, and provider flexibility.

1.6.1 Development Frameworks and Dev Environment

- **FastAPI:** lightweight Python framework used to build high-performance asynchronous APIs for agents and platform services.

Figure 2: Logo of FastAPI

- **Next.js:** React-based framework used to build the web frontend and provide a modular interface for agents, workflows, and configurations.

Figure 3: Logo of Next.js

- **LangChain:** library used to construct LLM-driven agents, connect tools, and manage prompt-based application logic.

Figure 4: Logo of LangChain

- **LangGraph:** library used to define graph-based orchestration logic for stateful multi-agent workflows.

Figure 5: Logo of LangGraph

- **Jupyter Notebook:** interactive environment used during prototyping, experimentation, SLM-evals, and prompt engineering.

Figure 6: Logo of Jupyter Notebook

1.6.2 LLM and SLM Providers

- **OpenAI:** commercial LLM provider used to access GPT-series models for high-quality generation and evaluation tasks.

Figure 7: Logo of OpenAI

- **Groq:** commercial inference provider used to access fast LLM serving, including Llama-based models.

Figure 8: Logo of Groq

- **Microsoft Phi-4:** small language model considered for cost-efficient and potentially self-hosted inference.

Figure 9: Logo of Microsoft Phi-4

- **Meta Llama :** open-weight LLM used as an alternative provider option for flexible deployment and benchmarking.

Figure 10: Logo of Meta Llama

- **SLM-evals:** used to benchmark providers and models in order to choose the best mix for the VIP provider layer.

1.6.3 Database, Storage, and Experiment Tracking

- **PostgreSQL:** relational database used to store configurations, metadata, workflows, and other structured platform data.

Figure 11: Logo of PostgreSQL

- **MLflow:** experiment tracking and artifact registry used to version models, prompts, schemas, datasets, and evaluation metrics.

Figure 12: Logo of MLflow

1.6.4 Deployment and Containerization

- **Docker:** container platform used to package individual agents, the builder, the orchestrator, and supporting services into reproducible images.

Figure 13: Logo of Docker

- **Docker Compose:** used during development and testing to run multi-service setups with consistent networking and configuration.

Figure 14: Logo of Docker Compose

- **GitLab:** used for source code hosting, version control, collaboration, and continuous integration workflows during development.

Figure 15: Logo of GitLab

1.6.5 Observability and Monitoring

- **Laminar:** used to capture execution traces, token usage, latency, and cost metrics across agents.

Figure 16: Logo of Laminar

1.7 Methodology

1.7.1 Methodological Approaches Considered

Several methodological approaches were considered in order to structure the development of the VIP platform. The project combines software engineering, data science, agent lifecycle management, orchestration, deployment, and observability. Therefore, the selected methodology had to support both iterative development and long-term architectural control. The approaches considered were Scrum, IBM Data Science Methodology, and IBM Agent Development Lifecycle (ADLC).

1.7.2 Scrum Agile Methodology

Scrum is an Agile framework based on iterative and incremental development. The official Scrum Guide describes Scrum as an iterative and incremental approach used to optimize predictability and control risk. It supports progressive delivery by organizing work into short cycles, reviewing progress regularly, and adapting the next development steps according to feedback.

Scrum was considered because the project required frequent adjustments, progressive development, and regular validation of features such as agent creation, workflow orchestration, prompt management, and observability dashboards.

However, Scrum alone was not sufficient for this internship because the project required more than sprint-based feature delivery. It also required a structured lifecycle for AI/data-related work, architectural traceability, governance, deployment control, and continuous monitoring of AI agents. For this reason, Scrum-inspired practices were retained for implementation organization, but Scrum was not selected as the main project methodology.

1.7.3 IBM Data Science Methodology – The Foundational Methodology

The IBM Data Science Methodology was considered as the foundational methodology because it provides the macro lifecycle of the project. The CognitiveClass data science methodology is structured around phases that move from business understanding and analytic approach to data requirements, collection, understanding, preparation, modeling, evaluation, deployment, and feedback. This structure makes it suitable for organizing an AI-oriented project from problem definition to operational feedback.

The methodology consists of 10 stages organized in a circular workflow:

- **Business Understanding:** defining the problem to be solved and establishing the main objectives.
- **Analytic Approach:** determining the appropriate analytical and technical approach to address the problem.
- **Data Requirements:** identifying the information and resources required to address the problem.
- **Data Collection:** gathering the required data and project inputs from the relevant sources.
- **Data Understanding:** exploring and examining the available information in order to understand quality, constraints, and relationships.
- **Data Preparation:** cleaning, transforming, and formatting the required data or resources for later use.
- **Modeling:** building predictive, descriptive, or intelligent models using appropriate algorithms and implementation choices.
- **Evaluation:** assessing the obtained results against the initial objectives and expected quality criteria.
- **Deployment:** implementing the solution in a production or operational environment.
- **Feedback:** monitoring performance, collecting feedback, and iterating on earlier stages when improvements are required.

Figure 17 presents the original IBM Data Science Methodology lifecycle, which was used as the main macro-framework for structuring the project from business understanding to deployment and feedback.

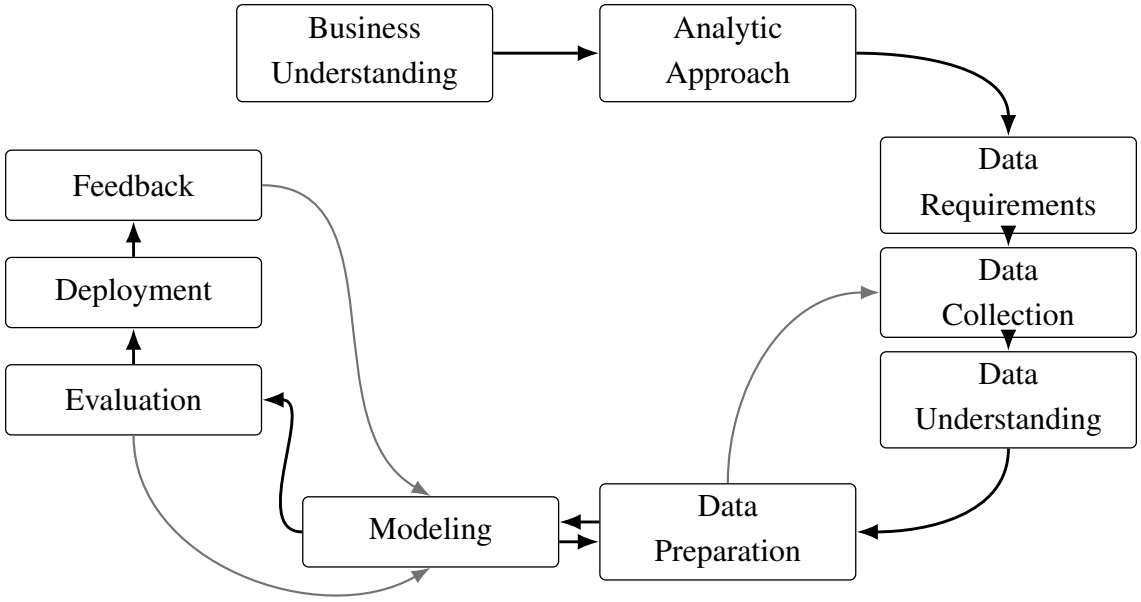

Figure 17: Original IBM Data Science Methodology lifecycle

This methodology is relevant for the VIP project because it supports a structured understanding of the business problem, the identification of requirements, the preparation of resources, the evaluation of outputs, and the deployment of the final solution. It also provides a feedback loop, which is important for improving agent behavior, model selection, and workflow reliability over time.

1.7.4 IBM Agent Development Lifecycle

While the IBM Data Science Methodology provides the overall project structure, it does not fully describe the specific lifecycle of AI agents. The VIP platform required a more agent-specific perspective because agents are generated from configuration, connected to tools, deployed as services, monitored, and improved after execution. For this reason, ADLC was used as a complementary framework.

IBM defines ADLC as a structured lifecycle for building, deploying, optimizing, and managing AI agents through distinct phases and iterative loops. In the context of VIP, this perspective is relevant because agents must remain reliable, observable, auditable, and aligned with business requirements after deployment.

In this report, the ADLC perspective is summarized through the following phases:

- **Plan:** alignment of stakeholders, use cases, objectives, success metrics, expected agent behavior, and operating procedures.
- **Code & Build:** implementation of agents through model selection, prompt design, tool integration, orchestration logic, version control, and controlled development environments.
- **Test & Release:** evaluation of agents against predefined criteria, including policy checks, security validation, and release readiness before cataloging or production use.

- **Deploy:** controlled rollout of validated agents into production environments with governance, versioning, rollback, and runtime security mechanisms.
- **Operate:** continuous observation and optimization of deployed agents using operational metrics such as accuracy, latency, cost, and user satisfaction.
- **Monitor:** ongoing auditing of agents and tools to support fairness, transparency, compliance, observability, and reproducibility.

Figure 18 illustrates the IBM Agent Development Lifecycle, which was used to guide the agent-specific phases of the project.

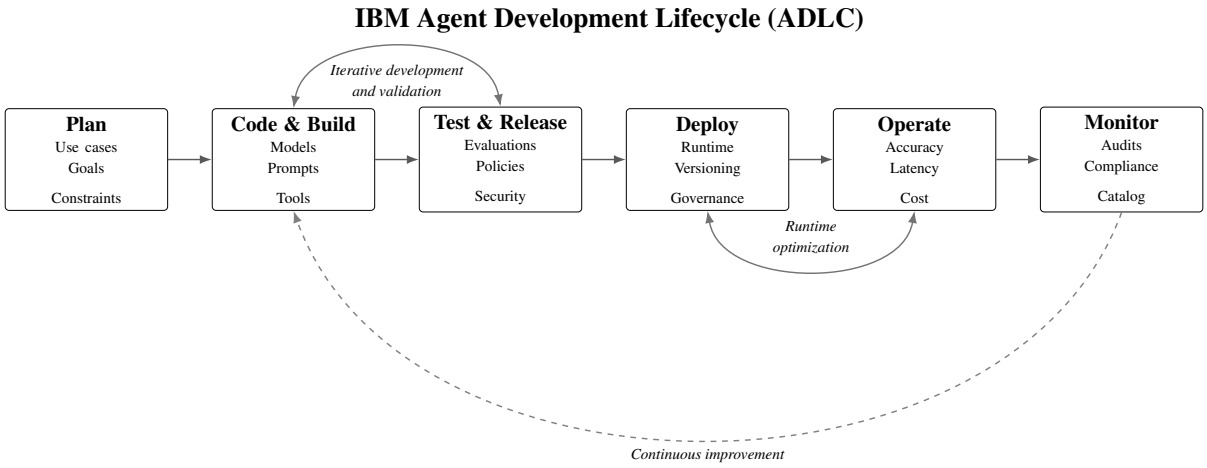

Figure 18: IBM Agent Development Lifecycle (ADLC)

In the VIP project, the Plan phase corresponds to defining agent use cases, expected behavior, schemas, and constraints. The Code & Build phase corresponds to configuring prompts, tools, models, and runtime artifacts. The Test & Release phase corresponds to validating agent outputs and schema compliance. The Deploy phase corresponds to packaging agents as Dockerized FastAPI services. The Operate and Monitor phases correspond to observing latency, token usage, cost, and execution traces.

IBM also describes agent building as the transformation of intent and design into concrete capabilities by configuring models, behavior, knowledge, and tools. This is aligned with the VIP approach, where agents are created from declarative configurations including prompts, schemas, tools, and model parameters.

1.7.5 Comparative Analysis of the Methodologies

The three methodologies were compared according to their relevance to the VIP project, especially with regard to flexibility, strategic vision, AI project support, agent lifecycle coverage, deployment, monitoring, and limitations.

Criterion	Scrum	IBM Data Science Methodology	IBM ADLC
Approach	Iterative and sprint-based	Structured and data/AI-oriented	Agent lifecycle-oriented
Flexibility	High	Medium to high	High
Strategic vision	Limited to product increments	Strong	Focused on agent operations
Fit for AI projects	Partial	Strong	Strong for agentic AI
Fit for agent lifecycle	Limited	Partial	Very strong
Deployment and monitoring support	Not detailed	Present through deployment and feedback	Central through deploy, operate, and monitor
Main limitation	Not enough governance for AI agents	Not specific enough for agent runtime behavior	Too agent-specific to structure the whole project alone

Table 1: Comparison of the methodological approaches considered for the VIP project

The comparison shows that none of the methodologies fully covers the needs of the VIP project when used alone. Scrum supports practical development organization, IBM Data Science Methodology provides the strategic project lifecycle, and ADLC addresses the agent-specific lifecycle required by configurable, deployable, and observable AI agents.

1.7.6 Adopted Hybrid Methodology for the VIP Project

Based on the comparison, the adopted methodology is a hybrid approach combining IBM Data Science Methodology and IBM ADLC, while keeping Scrum-inspired iterations for practical development organization.

IBM Data Science Methodology was selected as the main macro-framework because it structures the project from business understanding to deployment and feedback. It ensures that the project remains aligned with business objectives, technical constraints, and evaluation criteria.

ADLC was added because the VIP platform is centered on AI agents. The project required agent-specific phases such as planning agent behavior, configuring prompts and tools, validating outputs, deploying agents as services, operating them, and monitoring their execution.

Scrum was not selected as the main methodology because the project required stronger architectural governance and lifecycle control than a sprint-based framework alone could provide. However, Scrum-inspired practices were useful during implementation, especially for breaking the work into incremental tasks, validating features progressively, and adapting to feedback.

This hybrid methodology was therefore chosen because it provides a balance between structure, flexibility, and agent-specific lifecycle management. It supports the strategic organization of the project, the iterative improvement of AI components, and the operational requirements of an enterprise-grade agentic platform.

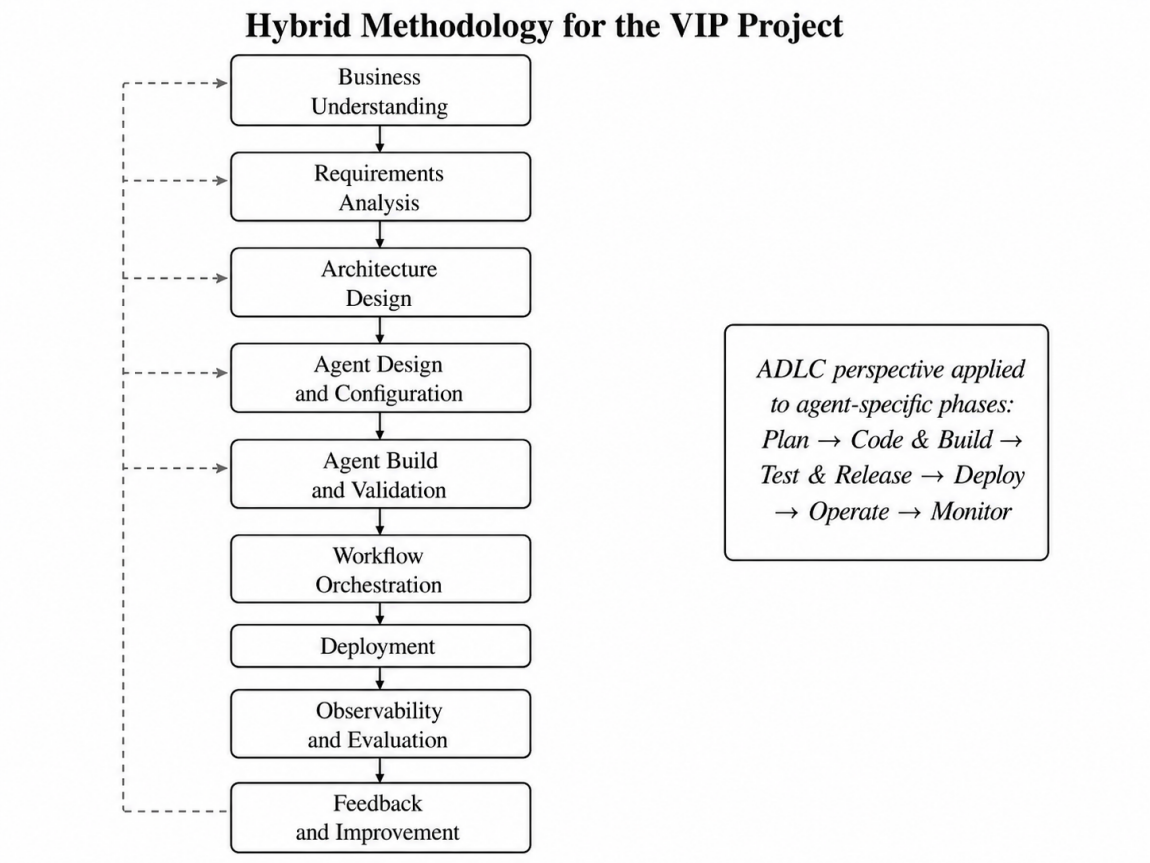

Figure 19: Hybrid methodology adopted for the VIP project

1.8 Project Timeline

The project officially started on December 1, 2025 and is scheduled to end on May 31, 2026, spanning a duration of six months. This timeframe was organized to support a structured and progressive approach, covering onboarding, research, design, development, integration, observability, and testing phases.

1.9 Summary

This chapter introduced the context of the internship by presenting the motivation behind enterprise AI agent orchestration, the host company Value Digital Services, and the objectives of the VIP project. It also clarified the main problem addressed by the project, namely the need for a modular, observable, cost-optimized, and provider-independent platform for creating and managing AI agents. The chapter then summarized the selected technologies, including FastAPI, Next.js, LangChain, LangGraph, Docker, MLflow, PostgreSQL, Laminar, and the LLM/SLM providers evaluated through SLM-evals, before presenting the methodological approach and the project timeline.

The next chapter focuses on the state of the art. It reviews existing agent frameworks, LLM providers, orchestration approaches, and evaluation methods in order to position the proposed solution with respect to current tools and practices.

Chapter 2

State of the Art

2.1 State of the Art of AI Agents and Orchestration

2.1.1 Defining AI Agents

An AI agent is a software entity capable of perceiving inputs, reasoning over context, invoking external tools, and producing structured outputs in order to achieve a defined objective. Unlike traditional prompt-response systems, agentic systems extend large language models (LLMs) with execution logic, memory, tool interfaces, and multi-step reasoning capabilities.

In enterprise environments, an agent is not merely a conversational component, but an operational unit that interacts with APIs, databases, and external systems. It may retrieve information, transform data, validate outputs, or trigger automated workflows. As a result, agents must be treated as managed execution components rather than isolated prompt wrappers.

This evolution from simple generation models to operational agents introduces new requirements: lifecycle management, configuration control, structured outputs, monitoring, and governance.

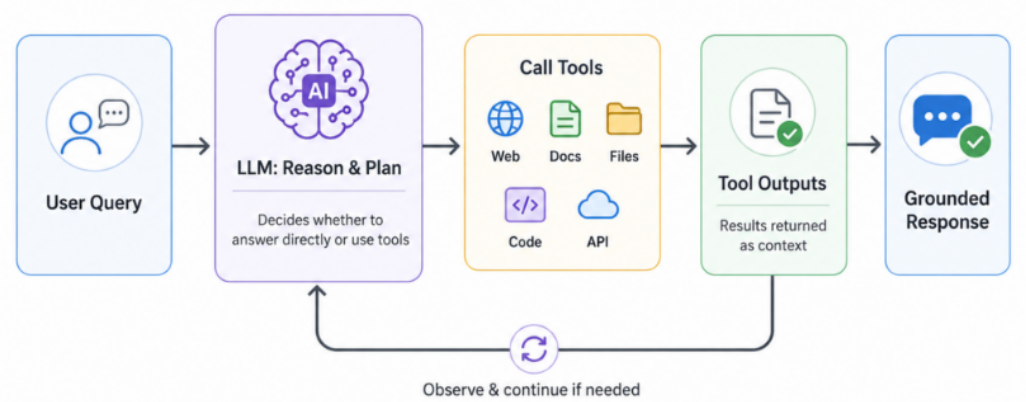

Figure 20: AI Agent Architecture.

2.1.2 Structure of AI Agents

AI agents can either be single or multi depending on the design structure.

Single Agent vs Multi-Agent

A single agent is an agent design where one LLM handles the whole task. It reasons, plans, and calls the required tools when needed. Most AI agents start as single-agent systems because they are simpler, easier to maintain, and usually enough for many tasks.

A multi-agent system uses specialised agents to solve different parts of a task. It often has a central agent, usually called an **orchestrator**, **supervisor**, or **planner**, that coordinates the other agents and decides when each one should act. Each specialised agent can have its own role, tools, and reasoning logic, making the system more modular and suitable for complex workflows.

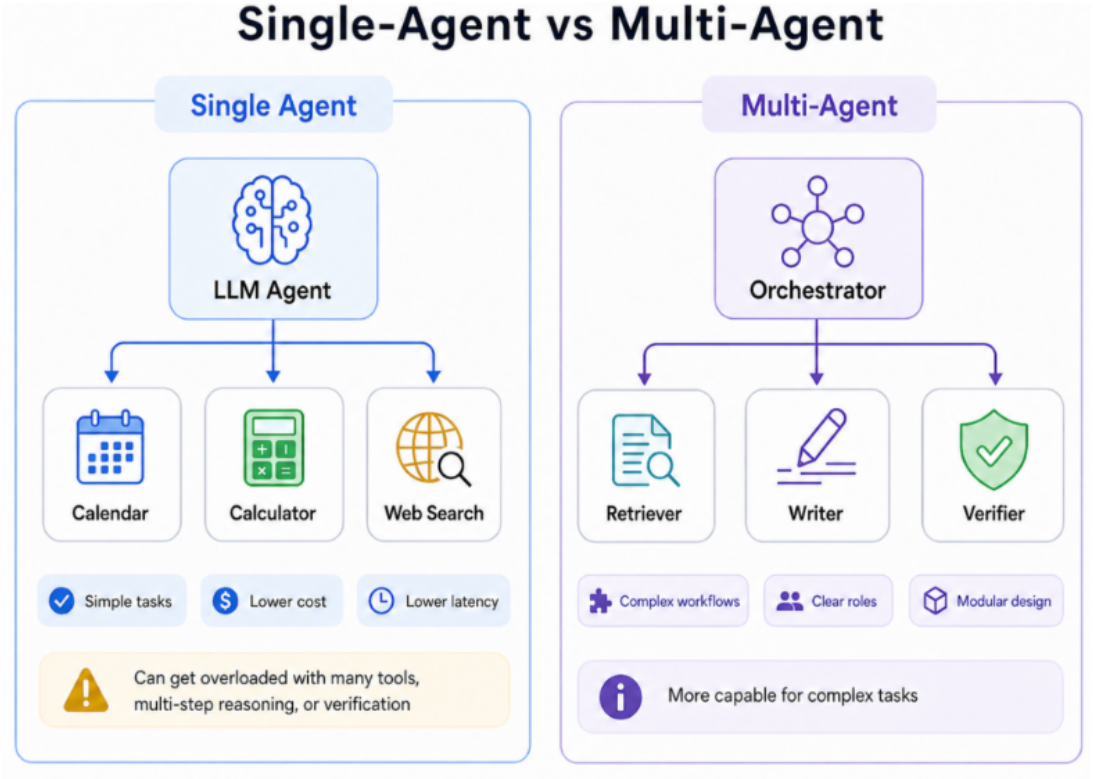

Figure 21: Single-Agent vs Multi-Agent.

When to Build a Multi-Agent System

A single-agent design works well for simple tasks that require limited tool use. For example, a personal assistant agent that can access a calendar to book reminders, a calculator agent that only uses a calculator tool, or a web search agent that uses a web search API to retrieve up-to-date information.

However, a single agent can become overloaded when the task requires many tools, multi-step reasoning, different responsibilities, or verification before the final response is returned to the user. Common issues include overloaded prompting, poor tool routing, unclear agent responsibilities, and reduced reliability due to too much complexity in one agent.

A **multi-agent system** is a better choice when the task may overwhelm a single-agent design and when specialised agents with clear roles, their own tools, and separate responsibilities are needed.

For example, a **software engineering agent** may work better as a multi-agent system:

```
Orchestrator -> Coder -> Tester -> Reviewer
```

The **Orchestrator** coordinates the workflow, the **Coder agent** generates the code, the **Tester agent** checks whether the code works, and the **Reviewer agent** reviews the solution to check for missing parts or possible improvements.

Another example is a **research agent** that researches a topic, retrieves information from different data sources, and generates grounded content:

```
Orchestrator -> Retriever -> Writer -> Verifier
```

The **Retriever agent** gathers information from the web and local documents stored in a vector database. The **Writer agent** writes based on the retrieved content. The **Verifier agent** checks the written content for errors, citations, and factual accuracy before the final response is returned.

Multi-agent systems make the workflow more modular and give each stage a clear role. However, they should be used only when the task genuinely needs that design, because they usually increase latency, cost, and maintenance complexity due to more LLM calls and more moving parts.

A simple rule is:

> *Use a single agent when the task is simple, has fewer steps, and needs only a few tools. Use a multi-agent system when the task requires specialised roles, multi-step reasoning, stronger verification, or coordination across different tools and workflows.*

In this context, VIP supports both Single-Agent Systems (SAS) and Multi-Agent Systems (MAS). Simple use cases can be handled by one configurable agent, while complex workflows can be distributed across coordinated specialised agents with clear responsibilities.

2.2 Preliminary Study and Project Specifications of the VIP Project

2.2.1 Problem Statement

Despite the rapid evolution of agentic AI frameworks and language models, enterprise deployment remains fragmented. Existing solutions often provide partial capabilities, focusing either on workflow automation, agent collaboration, or managed cloud execution.

However, organizations require platforms that combine:

- Configuration-driven agent generation
- Provider flexibility
- Workflow orchestration
- Deployment automation

- Observability and governance

The central challenge addressed by VIP is therefore:

> How can we design a modular, observable, provider-neutral AI agent orchestration platform capable of generating and managing agents dynamically while maintaining cost control, scalability, and governance?

2.2.2 Business Objectives

The business objectives of VIP are:

- Reduce vendor lock-in by abstracting model providers
- Enable rapid agent creation without modifying source code
- Improve governance through versioning and traceability
- Reduce operational cost through modular deployment
- Provide reusable workflows for domain-specific AI applications

2.2.3 Technical Objectives

From a technical perspective, the VIP project aims to provide a modular platform for creating, deploying, orchestrating, and supervising AI agents. The platform must support configurable agent behavior, reusable workflow composition, provider abstraction, and structured communication between agents.

These objectives are not limited to model performance. They also concern the way agents are generated, connected, monitored, and maintained throughout their lifecycle. Therefore, the main technical objectives of VIP are to enable configuration-driven agent creation, standardize agent communication through schemas, support multiple LLM and SLM providers, and ensure traceability during execution.

The model evaluation aspect is supported by the SLM-evals second project, which is presented separately in Section 2.4.

The main technical objectives of VIP are:

- enable dynamic agent creation from declarative configuration files;
- support reusable multi-agent workflows;
- provide a unified provider abstraction layer;
- ensure structured input and output validation through schemas;
- support prompt and schema versioning;
- provide observability over agent execution, token usage, latency, and errors;
- allow future integration of model evaluation results into provider selection.

2.2.4 Functional and Non-Functional Requirements

2.2.4.1 Functional Requirements

- Dynamic agent creation from YAML/JSON configuration files
- Model, provider, temperature, and prompt selection
- Tool binding through a tools catalog
- Structured schema-based input and output validation
- Multi-agent workflow composition
- Visual dashboard for managing agents and workflows
- Versioning of prompts and schemas using MLflow

2.2.4.2 Non-Functional Requirements

- Scalability through containerized deployment
- Performance optimization via lightweight Docker images
- Observability of API calls, token usage, and latency
- Reliability through schema validation and structured outputs
- Maintainability through versioned configuration artifacts
- Provider independence

2.2.5 Success Criteria

VIP is considered successful if:

- Agents can be generated and deployed without modifying source code
- Multi-agent workflows execute reliably
- Execution traces are observable
- Provider switching can be performed without architectural modification
- Evaluation results guide provider selection decisions

2.3 Comparison of Existing Platforms

In order to position VIP with respect to existing agentic and LLM-based platforms, this section presents a comparative study of three representative solutions: n8n, CrewAI, and Dify. These platforms were selected because they represent three complementary approaches: visual workflow automation, code-first multi-agent orchestration, and LLM application development. The objective is not to identify a direct competitor to VIP, but rather to highlight the architectural gaps that justify the design choices adopted in this project.

The comparison is particularly relevant because VIP aims to go beyond simple workflow automation. Its architecture focuses on generating configurable agents, deploying them as independent services, orchestrating them through explicit contracts, and monitoring their execution.

2.3.1 Evaluation Criteria Derived from VIP Requirements

The evaluation criteria are derived from the main requirements of VIP. Since the platform is designed to build, deploy, and operate configurable AI agents, the comparison focuses on orchestration, agent modularity, provider flexibility, observability, deployment model, and governance. These criteria reflect the main technical choices of VIP, namely configuration-driven agent behavior, independent Dockerized agents, MLflow-based prompt and schema versioning, schema validation, and workflow orchestration.

Table 2: Evaluation criteria derived from VIP requirements

Criterion	**Why it matters for VIP**
Orchestration model	VIP needs to coordinate multiple agents in workflows, not only execute isolated prompts.
Agent abstraction	VIP treats each agent as a configurable and reusable unit.
Provider flexibility	VIP must support different LLM providers without rewriting the platform.
Observability	VIP needs traces, token usage, latency, and cost monitoring.
Deployment model	VIP uses independently deployable containerized agents.
Schema and contract control	VIP relies on structured inputs, structured outputs, and schema validation.
Scalability	VIP must scale orchestration and agent execution separately.
Ease of use	VIP should simplify agent creation without losing technical control.
Enterprise readiness	VIP targets controlled, auditable, and maintainable AI systems.

These criteria allow us to evaluate whether each platform is workflow-centric, agent-centric, or application-centric, and to identify which limitations VIP should address.

2.3.2 Comparative Study of n8n, CrewAI and Dify

2.3.2.1 n8n

n8n is a workflow automation platform based on a visual node-based editor. It allows users to connect services, APIs, and automation steps through graphical workflows. In the context of AI, n8n provides an AI Agent node. According to the official documentation, an AI agent in n8n receives data, makes decisions, and acts in its environment by using external tools and APIs. The documentation also specifies that at least one tool sub-node must be connected to the AI Agent node.

n8n is relevant to VIP because it demonstrates the value of visual workflow composition and low-code automation. However, it remains mainly workflow-centric. The main object in n8n is the workflow, while in VIP the main object is the agent lifecycle: configuration, generation, deployment, execution, monitoring, and versioning. n8n provides useful enterprise features such as source control, log streaming, security settings, and insights, and it also supports deployment and monitoring configurations such as Docker, queue mode, OpenTelemetry, and scaling options. Nevertheless, VIP requires a more explicit agent contract, stronger schema enforcement, and independent agent containers generated from configuration.

The main lesson for VIP is that n8n is strong for visual workflows, integration, and automation. VIP should therefore learn from its ease of use while remaining different in its core focus: agent generation, schema control, and containerized runtime management.

2.3.2.2 CrewAI

CrewAI represents the category of code-first multi-agent frameworks. It is designed around the concept of a crew, which is a collaborative group of agents working together to achieve a set of tasks. Each crew defines the strategy for task execution, agent collaboration, and the overall workflow. CrewAI supports attributes such as agents, tasks, process type, memory, callbacks, planning, and knowledge sources.

CrewAI is close to VIP from a multi-agent perspective because both approaches rely on task decomposition and specialized agents. CrewAI also supports YAML configuration as a recommended way to define crews, agents, and tasks, which is aligned with the configuration-driven philosophy of VIP. However, CrewAI is mainly a framework used inside a Python runtime, while VIP aims to provide a complete platform where agents are generated, containerized, registered, orchestrated, and monitored as independent services.

This distinction is important because VIP's architecture separates build-time and runtime responsibilities. The builder receives an agent specification and produces a standalone artifact containing runtime code, selected tools, dependencies, Dockerfile, compose file, and manifest. This makes VIP closer to an operational platform than to a pure development framework.

The main lesson for VIP is that CrewAI is strong for explaining multi-agent collaboration through agents, roles, tasks, and processes. VIP goes further by adding Dockerized independent services, MLflow governance, schema contracts, and orchestration over HTTP.

2.3.2.3 Dify

Dify represents the category of LLM application development platforms. Its official documentation describes it as an open-source platform for building agentic workflows. It allows users to define processes visually, connect tools and data sources, and deploy AI applications.

Dify is relevant to VIP because it focuses on simplifying the development of LLM-based applications through visual workflows and integration with external tools and data. Compared with n8n, Dify is more oriented toward AI applications. Compared with CrewAI, it is less code-first and more product-oriented. However, VIP has a more infrastructure-oriented vision: each agent is treated as a deployable unit with explicit input/output schemas, selected tools, MLflow-managed artifacts, and container-level isolation.

This difference is aligned with VIP's objective of reducing fragmented AI initiatives. The internal platform vision highlights the need to move away from isolated workflows, fragmented models, and unscalable deployments toward a unified and auditable agent-centric platform.

The main lesson for VIP is that Dify is strong for LLM application creation, visual configuration, and AI workflow building. VIP should keep more control over agent runtime, schemas, observability, and deployment while learning from Dify's platform-oriented user experience.

2.3.3 Comparative Synthesis and VIP Positioning

The following table summarizes the comparison between n8n, CrewAI, and Dify according to the criteria derived from VIP requirements. The last column identifies the implications for VIP and shows how the proposed platform can combine the strengths of existing solutions while addressing their limitations.

Table 3: Synthesis of existing platforms and VIP positioning

Criterion	**Observation from existing platforms**	**VIP implication**
Workflow design	n8n and Dify provide accessible visual workflow creation.	VIP should keep workflow creation simple while preserving technical control.
Multi-agent logic	CrewAI structures collaboration through agents, tasks, and crews.	VIP should support specialized agents coordinated by an orchestrator.
Agent lifecycle	Existing tools focus more on workflows or agent execution than full lifecycle management.	VIP should cover configuration, generation, deployment, execution, monitoring, and versioning.
Deployment	Deployment depends on each platform or surrounding infrastructure.	VIP should standardize deployment through Dockerized independent agents.
Governance	Governance and observability are present but not always central or customizable.	VIP should rely on MLflow, schema validation, traces, latency, token usage, and cost monitoring.
Positioning	Existing platforms solve parts of the problem but not the full VIP target.	VIP is positioned as an agent lifecycle and orchestration platform.

This comparison shows that VIP is positioned at the intersection of three approaches. From n8n, it can retain the idea of accessible workflow composition. From CrewAI, it can reuse the principle of specialized collaborative agents. From Dify, it can adopt the product-oriented view of LLM application development. However, VIP differentiates itself through its contract-first architecture, Dockerized agent-per-service model, MLflow-based governance, schema validation, adapter logic, and observability layer. These elements are necessary because the target system is not only an AI workflow tool, but a platform for building, deploying, and operating reusable AI agents in a controlled environment.

2.4 Preliminary Study of the SLM-Evals Second Project

The SLM-Evals project was introduced as a complementary study to the VIP platform. Its objective is to evaluate whether smaller and more cost-efficient language models can be used for specific business tasks, especially financial report generation and analysis. This study is important because VIP is designed to support multiple providers and models, meaning that model selection should not be based only on popularity, but also on measurable criteria such as quality, cost, latency, schema compliance, and deployment constraints.

2.4.1 What are language models?

A language model is an artificial intelligence model designed to understand, generate, and manipulate natural language. It learns patterns from large text datasets and uses these patterns to predict the most probable sequence of words based on a given input. Modern language models are usually based on neural networks, especially transformer architectures, which allow them to capture contextual relationships between words over long sequences.

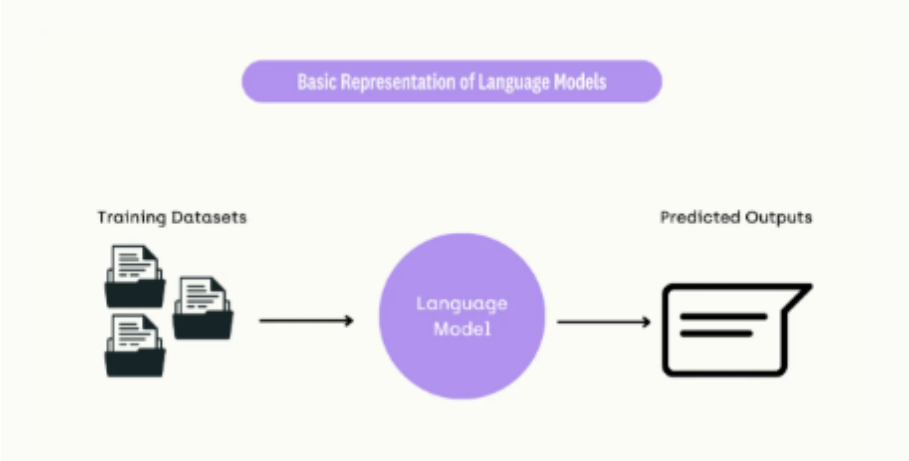

Figure 22: Basic representation of a language model.

2.4.2 What is an LLM?

Large Language Models, or LLMs, are language models trained on massive datasets and built with a very large number of parameters. This scale allows them to perform complex tasks such as text generation, summarization, translation, question answering, reasoning, and chatbot interaction. Examples include GPT models, Llama, Mistral, and Gemini-like models.

However, this high performance comes with important constraints. LLMs usually require significant computational resources for training and deployment, which increases infrastructure cost, energy consumption, and operational complexity. For this reason, they are powerful but not always the most efficient choice for every business use case.

2.4.3 What is an SLM?

Small Language Models, or SLMs, are lighter language models designed to offer a more resource-efficient alternative to LLMs. They usually contain fewer parameters and require less computational power, which makes them easier to deploy, faster to fine-tune, and more affordable to operate. They are especially useful when the task is narrow, well-defined, and domain-specific.

Unlike LLMs, SLMs are not necessarily designed to solve every possible language task. Their main value appears when they are adapted to a specific use case, such as document classification, structured information extraction, report generation, or domain-specific question answering. In business contexts, they can also reduce latency and support local or private deployment when sensitive data is involved.

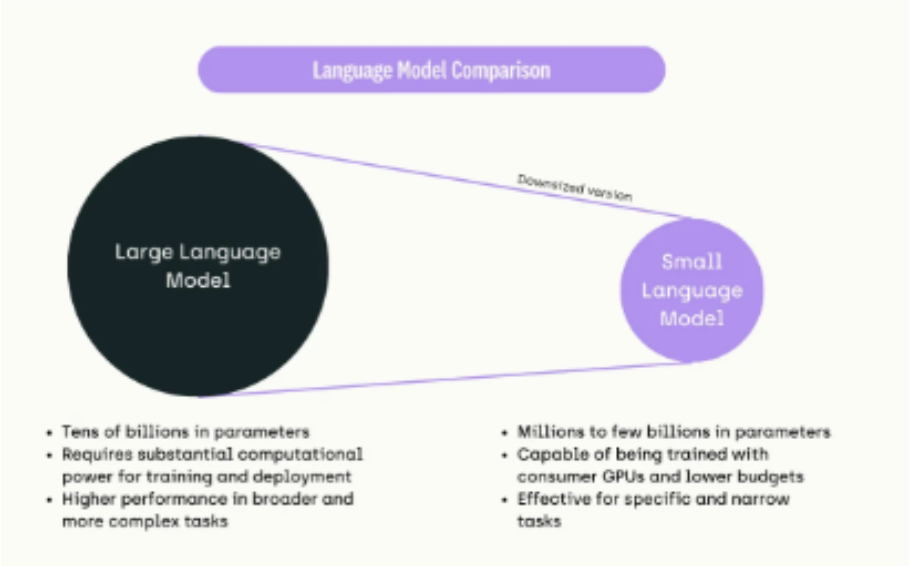

Figure 23: Comparison between large and small language models.

2.4.4 Motivation of the SLM-Evals Second Project

The motivation behind SLM-Evals is to determine whether smaller models can achieve acceptable quality for financial analysis tasks while reducing cost and latency. In the VIP context, this is important because each agent may not need the most powerful model available. For example, an extraction agent, a mapper agent, or a summarizer agent may perform well with a smaller model if the task is structured and supported by clear prompts and schemas.

This project therefore aims to compare models not only from a quality perspective, but also from an engineering and business perspective. A model that produces slightly lower quality but is cheaper, faster, and easier to deploy may be more suitable for some VIP agents. Recent

research on Phi-3, for example, shows that a 3.8B-parameter model can reach competitive results on benchmarks such as MMLU and MT-Bench while remaining small enough for local or lightweight deployment scenarios.

Therefore, the objective is not to prove that SLMs are always better than LLMs, but to identify when they are sufficient for a given task.

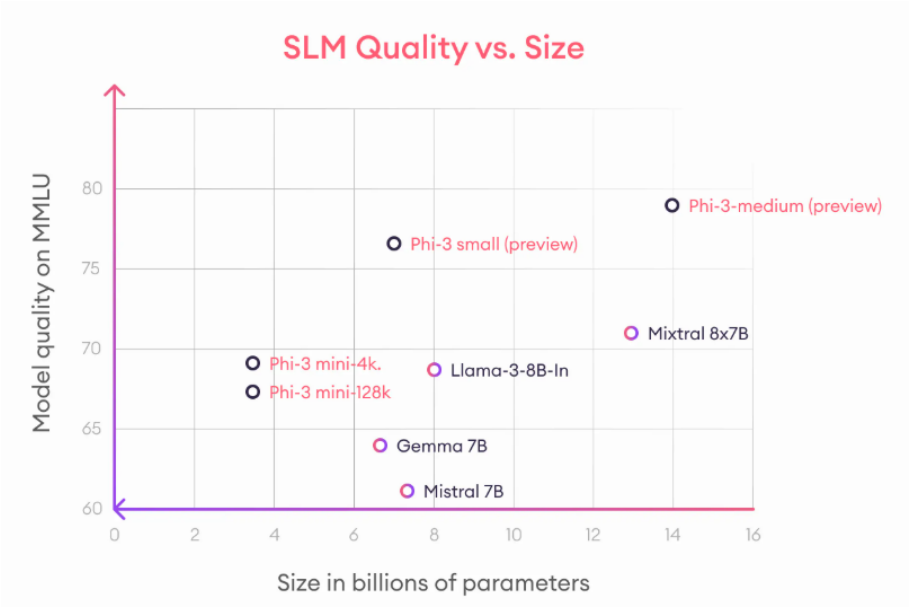

Figure 24: Quality and efficiency considerations for small language models.

2.4.5 Provider and Model Evaluation Need

Since VIP is designed to support multiple models and providers, model choice must be based on evaluation rather than intuition. Different providers may offer different trade-offs in terms of output quality, response time, token cost, availability, context length, and structured output support. For this reason, SLM-Evals provides a controlled evaluation process that helps compare models under the same task conditions.

For the VIP platform, the most relevant evaluation criteria are:

Table 4: Evaluation criteria for provider and model selection in VIP

Criterion	Why it matters for VIP
Output quality	Measures whether the generated financial analysis is correct and useful
Latency	Important for responsive agent workflows
Cost	Helps reduce operational expenses when many agents are executed
Schema compliance	Ensures that agents return structured and valid outputs
Provider flexibility	Avoids dependency on a single LLM provider
Stability	Ensures consistent behavior across repeated executions
Domain relevance	Measures how well the model handles financial language and reasoning

The results of this evaluation will support VIP's provider-selection strategy. Instead of assigning the same model to all agents, VIP can select the most appropriate model depending on the agent role, task complexity, and expected quality level. This aligns with the platform's objective of building configurable, cost-aware, and provider-independent AI workflows.

2.5 Chapter Summary

This chapter presented the state of the art related to the VIP project and introduced the main concepts required to understand the proposed platform. It first defined AI agents as operational software entities capable of reasoning, using tools, interacting with external systems, and producing structured outputs. The chapter also distinguished between single-agent and multi-agent systems, showing that single-agent architectures are suitable for simple tasks, while multi-agent systems are more appropriate for complex workflows requiring specialization, coordination, and verification.

The chapter then introduced the preliminary study and specifications of the VIP project. The main problem identified is the fragmentation of existing enterprise AI solutions, which often lack configuration-driven agent generation, provider flexibility, automated deployment, orchestration, observability, and governance. Based on these limitations, the business objectives, technical objectives, functional requirements, non-functional requirements, and success criteria of VIP were defined.

A comparative study of existing platforms was also conducted through the analysis of n8n, CrewAI, and Dify. This comparison showed that each platform addresses part of the problem: n8n provides strong visual workflow automation, CrewAI focuses on multi-agent collaboration, and Dify supports the development of LLM-based applications through agentic workflows. However, none of these solutions fully covers the specific needs of VIP, especially regarding independent agent containers, schema-based contracts, MLflow-based artifact governance, provider abstraction, and built-in observability. This analysis helped position VIP as a platform that combines workflow usability, multi-agent modularity, and infrastructure-level control.

Finally, the chapter introduced the SLM-Evals second project, which supports VIP by eval-

uating language models and providers according to practical criteria such as output quality, latency, cost, schema compliance, stability, and domain relevance. The distinction between LLMs and SLMs highlighted the importance of selecting models according to the complexity of each agent task instead of relying on a single model for all use cases. This evaluation perspective prepares the next chapters, where the architecture, implementation, orchestration, and evaluation of the VIP platform are presented in detail.

Chapter 3

System Design and Architecture

3.1 Introduction

This chapter presents the system design and architecture of VIP. Based on the limitations and requirements identified in the previous chapter, the objective is to describe how the platform is structured to support configurable agent generation, multi-agent orchestration, provider flexibility, deployment automation, and observability. The chapter focuses on the architectural organization of the platform, its main components, their responsibilities, and the interactions between them.

3.2 Architectural Vision of VIP

3.2.1 From Fragmented AI Solutions to an Agentic Hub

Enterprise AI initiatives often start as isolated prototypes, where each use case has its own prompts, tools, provider configuration, deployment logic, and monitoring approach. As these initiatives grow, this fragmentation creates duplicated effort, weak governance, limited observability, and higher maintenance complexity.

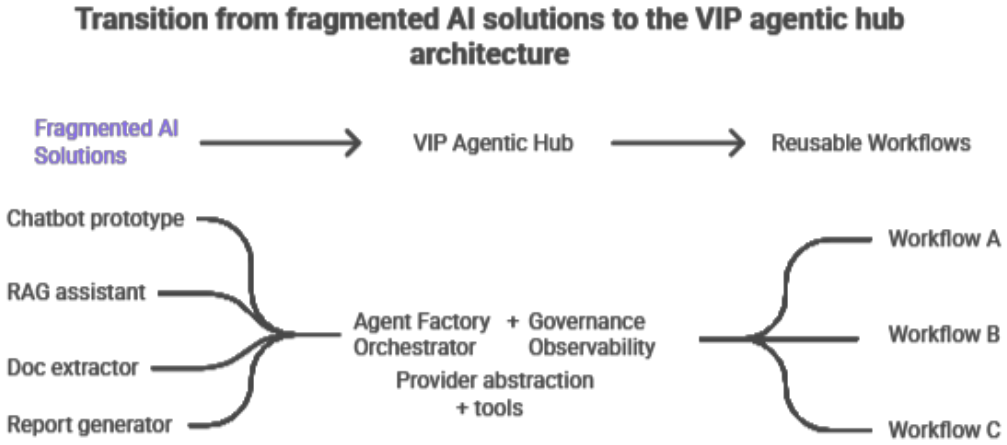

Figure 25: Transition from fragmented AI solutions to the VIP agentic hub

VIP responds to this problem through an agentic hub architecture. The platform centralizes agent creation, deployment, orchestration, provider configuration, schema validation, and

monitoring. This makes agents reusable, configurable, and observable components that can be combined into controlled business workflows.

3.2.2 Main Architectural Principles

The architecture of VIP is guided by a set of principles derived from the limitations identified in the previous chapter. These principles ensure that the platform remains modular, configurable, observable, and suitable for enterprise-level AI workflows.

Table 5: Main architectural principles of VIP

Principle	**Meaning in VIP**
Modularity	Each agent has a specific responsibility and can be reused in different workflows.
Configuration-driven behavior	Agent behavior is controlled through prompts, tools, schemas, model parameters, and provider configuration.
Zero code change	New agent behaviors can be created by changing configuration artifacts instead of modifying the agent source code.
Contract-first communication	Inputs and outputs are validated using schemas to make communication between agents more reliable.
Containerized deployment	Each generated agent can run as an independent Dockerized service.
Provider flexibility	The platform can use different LLM and SLM providers without being tied to one model provider.
Observability by design	Executions must be traceable through logs, latency, token usage, cost, and runtime metrics.

These principles define the architectural direction of VIP. Instead of designing agents as isolated scripts, the platform treats them as configurable, deployable, and observable services. This makes the system easier to extend, monitor, and adapt to different business workflows.

3.3 Global Architecture of the VIP Platform

This section presents the global architecture of the VIP platform. The objective is to describe how the platform interacts with its users and external systems, then to zoom into its main internal containers and components. The architecture is presented progressively using a context view, a container view, and a component summary.

3.3.1 System Context View

At the context level, VIP is represented as a single platform interacting with one main actor, the platform user. This user can create configurable agents, compose workflows from generated agents, execute these workflows, and consume the generated outputs. External systems such as LLM/SLM providers, MLflow, PostgreSQL, the observability layer, and the Docker environment support the platform execution.

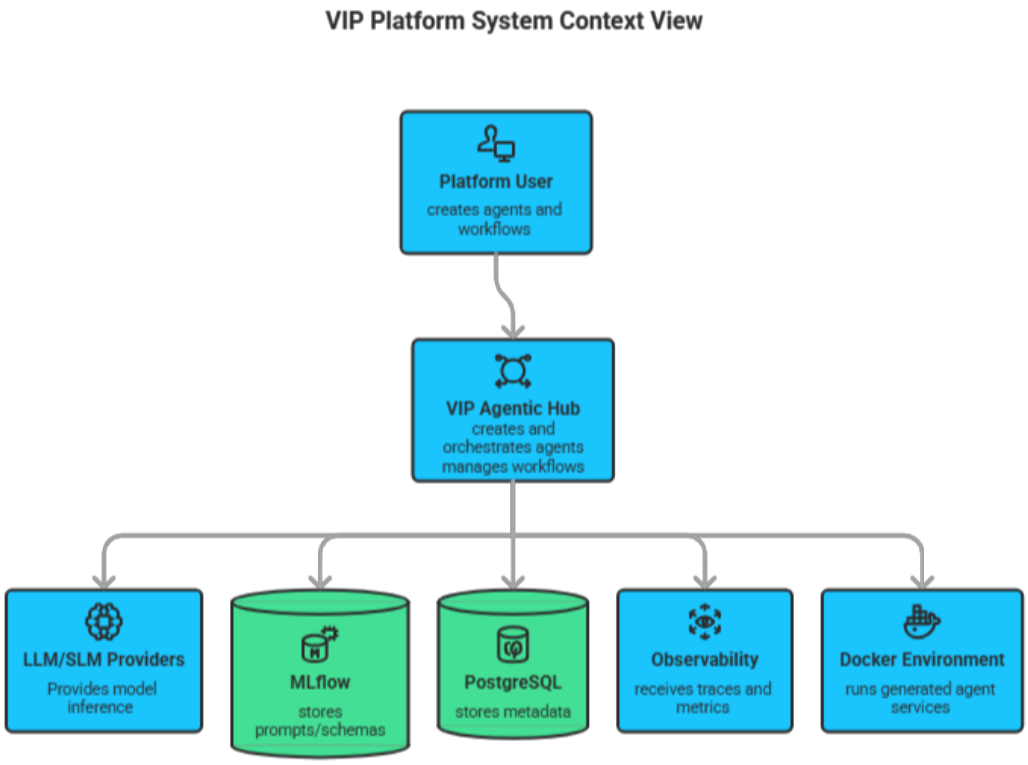

Figure 26: System context view of the VIP platform

3.3.2 Container View

After presenting the external context of VIP, the container view provides a more detailed representation of the main software units that compose the platform. At this level, VIP is no longer considered as a single black-box system. Instead, it is decomposed into its principal containers and runtime services, including the frontend, backend API, Agent Factory, generated agents, orchestration layer, storage services, governance layer, and observability layer.

The architecture is organized around two main operational responsibilities. The first responsibility is the creation of runnable agent services from declarative configurations, handled by the Agent Factory. The second responsibility is the coordination of generated agents into direct executions or multi-step workflows, handled by the orchestration layer. The Agent Core acts as the reusable runtime scaffold that defines the common behavior of all generated agents.

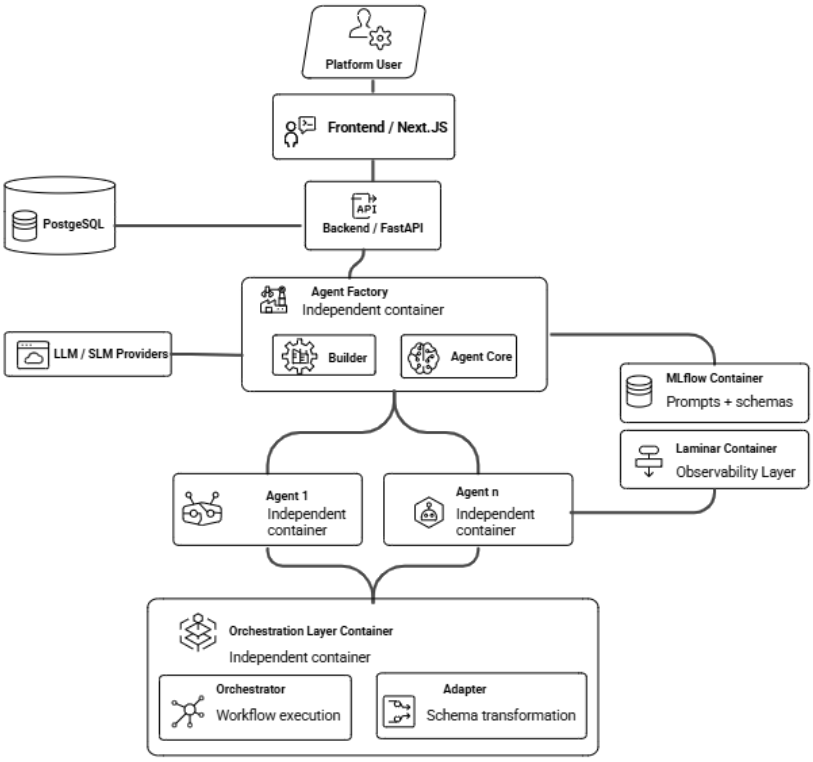

Figure 27: Container view of the VIP platform

This illustrates the main containers of the VIP platform. The platform user interacts with the system through the Next.js frontend, which communicates with the FastAPI backend. The backend coordinates agent creation through the Agent Factory and workflow execution through the orchestration layer. During agent creation, the selected provider, model, prompts, schemas, and tools are included in the generated agent configuration. At runtime, the generated agents operate as independent containers and can be orchestrated through the workflow execution layer. PostgreSQL stores platform metadata, MLflow manages prompts and schemas, Laminar collects observability data, and LLM/SLM providers are used for model inference.

3.3.3 Main Platform Components

The container view highlights the main components involved in the VIP platform. Each component has a specific responsibility in the agent lifecycle, from user interaction and configuration to agent generation, workflow execution, storage, governance, and monitoring. Table 6 summarizes the role of each component before presenting the detailed architecture of the main modules in the following sections.

Table 6: Main components of the VIP platform

Component	Responsibility
Platform User	Creates agents, composes workflows, executes them, and consults outputs.
Frontend / Next.js	Provides the interface for agents, workflows, prompts, schemas, tools, and monitoring.
Backend API / FastAPI	Coordinates requests between the frontend and platform services.
Agent Factory	Generates independent agent services from declarative configurations.
Agent Core	Provides the reusable runtime scaffold shared by generated agents.
Generated Agents	Execute specialized tasks as independent containers.
Orchestration Layer	Coordinates direct execution and multi-agent workflows.
Adapter	Transforms payloads between agents when schemas differ.
PostgreSQL	Stores platform metadata such as agents, tools, workflows, and configurations.
MLflow	Versions prompts, schemas, and related artifacts.
Laminar	Collects traces, latency, token usage, cost, and execution metrics.
LLM / SLM Providers	Provide model inference during agent execution.

3.4 Agent Lifecycle Architecture

3.4.1 Configuration-Driven Agent Creation

In VIP, an agent is not created by manually writing a new service for each use case. Instead, it is generated from a declarative configuration that describes how the agent should behave at runtime. This configuration defines the selected provider, the model, the temperature, the system and task prompts, the tools, the expected input schema, and the expected output schema.

This approach separates the reusable execution logic from the agent-specific behavior. The Agent Core provides the common runtime structure, while the configuration specializes this structure for a particular task. As a result, the same Agent Core can produce different agents without modifying the source code of the runtime itself. This supports the zero-code-change principle: creating or updating an agent mainly requires changing its configuration, not rewriting its implementation.

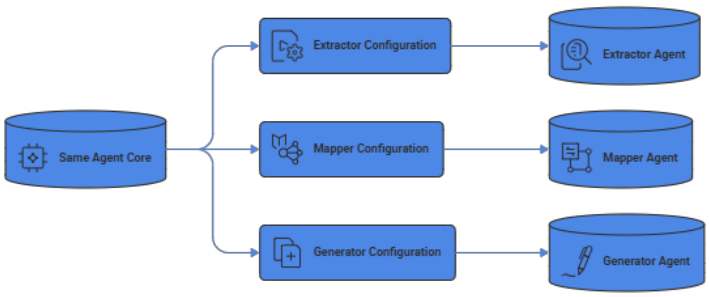

Figure 28: Configuration-driven agent specialization

3.4.2 Agent Generation Pipeline

The generation process starts when an agent configuration is sent to the Agent Factory. The Agent Factory first validates the configuration to ensure that the selected provider, model parameters, prompts, tools, schemas, and deployment metadata are complete and consistent. Once the configuration is accepted, the factory creates a generated agent artifact.

This artifact contains all elements required to execute the agent as an independent service. It includes the runtime code, the selected tools, the required dependencies, the Dockerfile used for containerization, and the configuration files required to run the agent as an independent service. After the artifact is produced, Docker builds the agent container. The generated agent then becomes a runnable service that can be deployed, called directly, or integrated into an orchestrated workflow.

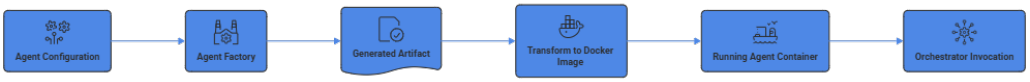

Figure 29: Agent generation and containerization pipeline

3.4.3 Agent Runtime Contract

Once generated and deployed, each agent exposes a standard runtime contract. This means that all agents can be invoked in a consistent way, regardless of their internal task or configuration. In VIP, a generated agent receives a structured input payload, validates it against its input schema, executes its configured logic, and returns a structured output that conforms to its output schema.

This common contract is important because generated agents are deployed as independent services. The orchestration layer does not need to know the internal implementation of each agent. It only needs to know the agent endpoint, its input schema, and its output schema. This allows the orchestrator to execute agents individually or combine them into multi-step workflows.

The runtime contract can be summarized as follows:

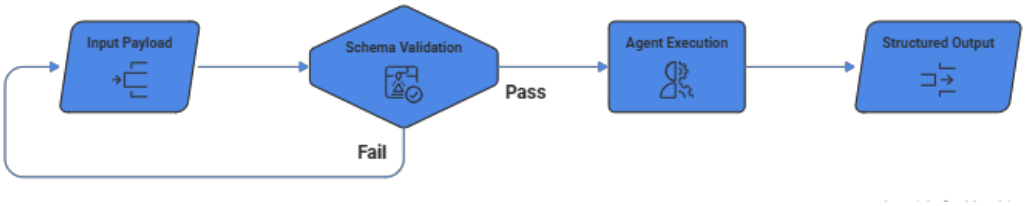

Figure 30: Standard runtime contract of a generated agent

During workflow execution, the output of one agent may become the input of another agent. If the output schema of a previous step does not directly match the input schema expected by the next step, the adapter is used to transform the payload into a compatible structure. This ensures that agents can remain specialized and independent while still being combined into coherent workflows.

Therefore, the runtime contract acts as a common communication layer between generated agents and the orchestrator. It improves reliability, reduces integration complexity, and supports the contract-first architecture of the VIP platform.

3.4.4 Agent Lifecycle Summary

The agent lifecycle in VIP can be summarized as a controlled transformation from configuration to execution. The process starts when the platform user defines the agent behavior through a declarative configuration. This configuration specifies the model provider, model parameters, prompts, tools, and input/output schemas. It is then processed by the Agent Factory to produce a standalone generated artifact.

This artifact is packaged as a Docker image and executed as an independent agent container. Once running, the agent can be invoked by the orchestration layer either directly or as part of a multi-step workflow. During execution, observability mechanisms collect traces, latency, token usage, cost, and execution metrics to support monitoring and later improvement.

This lifecycle establishes a clear separation between agent creation, runtime execution, and workflow coordination. The next section details the internal architecture of the Agent Factory and explains how the platform generates reusable and deployable agent artifacts.

3.5 Agent Factory Architecture

The Agent Factory is the build-time component responsible for transforming an agent configuration into a standalone and deployable runtime artifact. It plays a central role in VIP because it allows the platform to create specialized agents without modifying the source code of the runtime engine.

The main objective of the Agent Factory is to separate agent creation from agent execution. The platform user defines the desired behavior of the agent through configuration, while the Agent Factory handles the preparation of the runtime scaffold, selected tools, dependencies, Docker-related files, and metadata required for deployment and later orchestration.

3.5.1 Role of the Agent Factory

The Agent Factory is responsible for generating runnable agent services from declarative configurations. It does not execute the agent itself. Instead, it prepares all the elements required to build and run the agent as an independent service.

This component enforces a clear separation of responsibilities. The Agent Core remains the source of truth for runtime behavior, while the Agent Factory is responsible for assembly behavior. In other words, the runtime logic is reused across generated agents, but each generated agent can behave differently depending on its configuration, prompts, schemas, tools, model provider, and model parameters.

By centralizing the generation process, the Agent Factory ensures that all generated agents follow the same architectural rules. This improves consistency, reduces duplication, and supports the zero-code-change principle, where new agent behaviors are created by changing configuration artifacts rather than modifying the runtime source code.

3.5.2 Agent Configuration Input

The generation process starts with an agent configuration. This configuration describes the main properties required to create the agent, including its name, model provider, model name, temperature, prompt reference, memory behavior, input schema, output schema, and selected tools.

The configuration acts as the source of truth for the generated agent. It determines which model will be used, which prompts and schemas will be loaded from MLflow, and which tools will be included in the generated runtime. This allows the same Agent Core to be reused for different agent types such as extractor, mapper, generator, or sanitizer agents.

This approach makes the platform flexible because changing the configuration can produce a different agent behavior without changing the underlying runtime implementation.

The following example illustrates a simplified `config.yaml` file used to define an agent in VIP.

```
agent_name: extractor_agent
provider: openai
model: gpt-4o-mini
temperature: 0
prompt_uri: mlflow://prompts/extractor_v1
input_schema: mlflow://schemas/extractor_input
output_schema: mlflow://schemas/extractor_output
tools:
  - pdf_extractor
  - table_parser
```

This example shows how the behavior of an agent can be defined through configuration elements instead of source code modifications.

3.5.3 Tool Selection Mechanism

The Agent Factory includes only the tools selected in the agent configuration. Instead of copying the complete tool catalog into every generated agent, the builder selects the required tool files and stages them inside the generated artifact.

This mechanism ensures that each generated agent contains only the capabilities required for its task. For example, an extractor agent may include document parsing tools, while a mapper agent may include transformation tools. A generator agent may rely mainly on prompts and schemas, with fewer external tools.

The tool selection mechanism improves modularity and control. It reduces unnecessary dependencies inside generated agents and limits the actions that an agent can perform at runtime. This is important in an agentic platform because tools define the operational capabilities of an agent.

3.5.4 Dependency and Package Resolution Strategy

To avoid generating oversized agent containers, the Agent Factory applies a selective dependency and package resolution strategy. Instead of copying a complete shared dependency environment into every generated agent, the platform combines the common runtime requirements with only the requirements associated with the selected tools.

The same principle is applied to operating system packages. Each tool can define its own required system packages, and only the packages linked to the selected tools are included in the generated artifact. As a result, each generated agent receives a tailored `requirements.txt` and `apt-runtime-packages.txt` file.

This strategy makes generated agents lighter, more understandable, and easier to maintain. It also supports the modular architecture of VIP, because each agent contains dependencies that reflect its actual role instead of depending on the full platform tool catalog.

3.5.5 Generated Agent Runtime Structure

After the configuration, tool selection, and dependency resolution steps are completed, the Agent Factory prepares the runtime structure of the generated agent. This structure is not a complete copy of the whole VIP repository. Instead, it contains only the elements required for the agent to run as an independent FastAPI service.

The generated runtime is mainly composed of the reusable Agent Core files, the agent configuration, the selected tools, dependency files, and Docker-related files. This allows each generated agent to keep the same execution structure while specializing its behavior through configuration.

A simplified generated agent structure can be represented as follows:

```
generated_agent/
|-- app.py
|-- agent.py
|-- agent_config.yaml
|-- api/
|-- core/
|-- observability/
|-- services/
|-- tools/
|-- requirements.txt
|-- apt-runtime-packages.txt
|-- Dockerfile
```

The `agent_config.yaml` file defines the specific behavior of the generated agent, while the runtime files provide the common execution logic. The `tools/` directory contains the common tool interface files and only the selected tools required by the agent. The `requirements.txt` and `apt-runtime-packages.txt` files define the Python and operating system dependencies needed to build the final container image.

This structure supports modularity because each generated agent contains only what it needs to execute its task. It also improves maintainability, since the Agent Core remains the reusable source of runtime behavior, while the generated agent structure represents the concrete runtime package used for deployment.

3.5.6 Runtime Metadata and Agent Registry

After an agent is generated and launched, the platform must keep track of it so that it can be displayed, monitored, and reused in workflows. In the current architecture, the complete generated runtime folder is not stored in the database. Instead, PostgreSQL stores lightweight metadata describing the created agent and its runtime state, such as the agent identifier, name, configuration snapshot, builder request snapshot, container identifier, image tag, endpoint information, and runtime status.

This separation distinguishes runtime files from platform metadata. The generated runtime files are used to build and run the Docker container, while PostgreSQL stores the information needed to list agents, count them, monitor their status, and associate them with workflows. As a result, the agent library can display created agents based on database records, while execution remains handled by independent Docker containers.

The Agent Factory architecture makes agent creation controlled, modular, and reusable. By separating configuration, runtime scaffold, selected tools, dependencies, and runtime metadata, VIP can generate independent agent services while preserving consistency across the platform. The next section focuses on the orchestration layer, which is responsible for invoking these generated agents individually or as part of multi-step workflows.

3.6 Orchestrator Architecture

After the Agent Factory generates and deploys independent agent services, the platform requires a runtime component capable of coordinating their execution. This responsibility is

handled by the orchestration layer, which connects generated agents into direct executions or multi-step workflows.

3.6.1 Role of the Orchestrator

The orchestrator is the runtime coordination component of VIP. Agents perform the actual task execution, while the orchestrator decides which agent should run, in which order, with which payload, and under which input/output contract.

When a workflow is triggered, the orchestrator prepares the current payload, applies schema handling, invokes the target agent through its internal endpoint, and collects the produced output. This design keeps agents independent and reusable, while allowing the platform to combine them into more complex workflows.

3.6.2 Workflow Definition

A workflow defines the ordered sequence of agents required to complete a higher-level task. In VIP, this sequence is not fixed by the architecture. It is defined by the workflow specification and can vary depending on the business use case.

For illustration, a document-processing workflow can be represented as follows:

$$\texttt{Agent 1} \rightarrow \texttt{Agent 2} \rightarrow \texttt{Agent 3} \rightarrow \texttt{Agent n}$$

In the implemented prototype, one representative workflow follows a document-processing pattern composed of extraction, mapping, generation, and sanitization steps. The implementation details of this workflow are presented in Chapter 4.

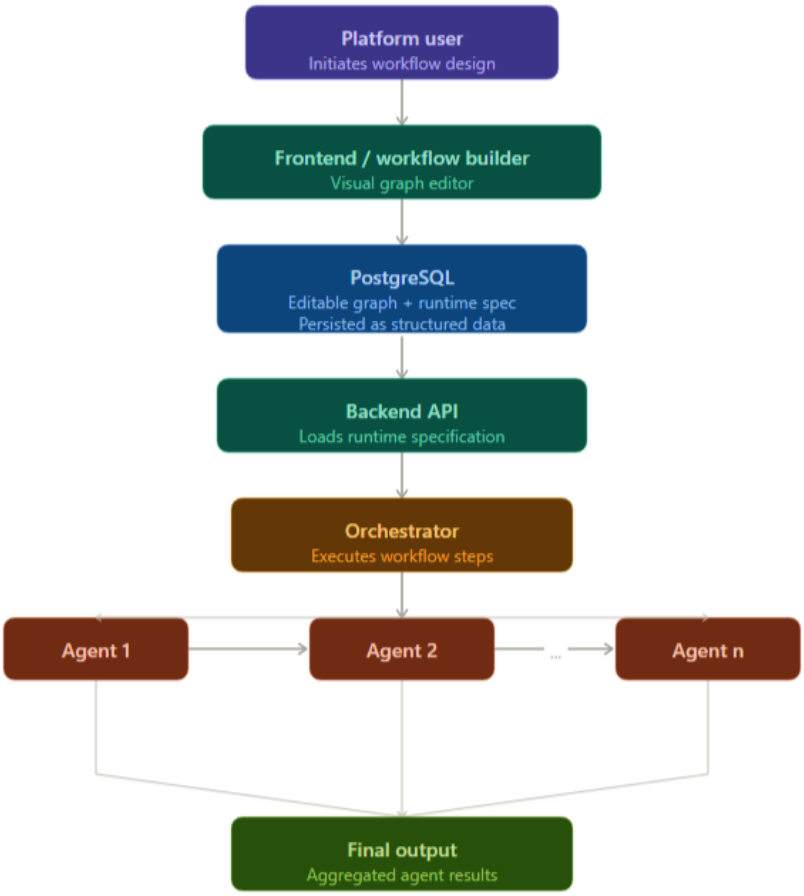

Figure 31: Workflow execution process in the VIP platform

Figure 31 shows the high-level workflow execution process in VIP. The backend loads the execution-ready workflow specification from PostgreSQL and sends it to the orchestrator. The orchestrator then invokes the generated agents step by step through their internal runtime endpoints until the final output is produced.

3.6.3 Schema-Based Communication

Because generated agents are independent services, communication between them must be controlled. VIP relies on schema-based communication to ensure that each workflow step receives a payload compatible with the input schema of the target agent.

Before invoking an agent, the orchestration layer checks whether the current payload can satisfy the target input schema. If the payload is already compatible, it can be passed directly to the agent. Otherwise, the adapter mechanism is used to reshape it before execution.

The communication process can be summarized as follows:

```
Current Payload → Schema Handling → AdapterAgent if Needed →
                  Agent Execution
```

3.6.4 Adapter Role in Workflow Execution

The adapter service is part of the workflow execution path before each agent call. It first attempts deterministic schema handling by checking whether the current payload already satisfies

the target input schema or can be made compatible through direct field matching and default values.

If this deterministic step is sufficient, the workflow continues without invoking the model-backed adapter. If the payload cannot be adapted directly, the `AdapterAgent` is used as a fallback to transform it into the expected structure. This decision depends on the actual payload compatibility with the next agent input schema, not only on whether two schema definitions look identical.

This mechanism improves modularity because agents do not need to be tightly coupled to each other. Each agent keeps its own input and output contract, while the orchestration layer manages compatibility between workflow steps.

3.7 Data, Storage, and Governance Architecture

The VIP platform relies on several storage and governance mechanisms to manage generated agents, workflows, prompts, schemas, provider choices, and runtime metadata. This layer is important because the platform does not only execute agents; it must also keep track of how agents are configured, how workflows are defined, and how runtime services are deployed.

In the current architecture, PostgreSQL is used as the main metadata storage layer, while MLflow is used for prompt and schema retrieval. Provider and model choices are stored as part of the agent configuration, and runtime metadata is persisted to support monitoring and reuse.

3.7.1 PostgreSQL Metadata Storage

PostgreSQL is used as the main operational metadata store of the VIP platform. It does not store the generated agent runtime files themselves. Instead, it stores structured metadata that allows the backend to manage applications, agents, reusable agent templates, and workflow definitions.

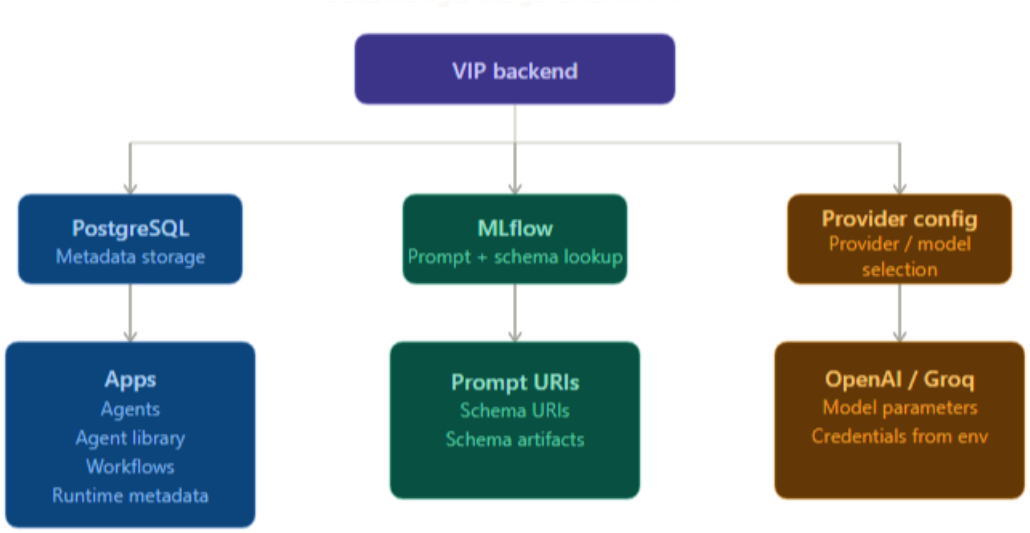

Figure 32: Data, storage, and governance architecture of the VIP platform

The main application tables are `apps`, `agents`, `agent_library`, and `workflow_definitions`. The `apps` table stores intelligent application or workspace records. The `agents` table stores

the agents created inside an application, including their configuration snapshots, selected tools, runtime status, container metadata, image tag, and endpoint information. The `agent_library` table stores reusable or prebuilt agent templates. Finally, the `workflow_definitions` table stores the workflows created through the user interface.

This storage layer acts as the application-level registry of the platform. It allows VIP to list created agents, count them, display their runtime state, associate them with workflows, and keep their configuration references. However, the generated runtime files, such as the agent source scaffold, `agent_config.yaml`, selected tool files, dependency files, and Docker image layers, remain outside PostgreSQL. They are part of the generated runtime environment and Docker container lifecycle.

3.7.2 MLflow-Based Prompt and Schema Management

MLflow is used in VIP as a registry and artifact lookup mechanism for prompts and schemas. Generated agents do not embed prompt or schema content directly inside their runtime code. Instead, they store references to these resources through URIs.

For prompts, the generated agent configuration contains a prompt URI. At runtime, the agent loads the prompt from MLflow using this reference. This makes it possible to change the prompt version used by an agent without modifying the reusable runtime code.

For schemas, VIP uses schema URIs to reference input and output contracts. The schema content is stored in MLflow as JSON artifacts and loaded by the agent runtime when needed. This supports schema-based validation and helps ensure that generated agents receive and produce structured data.

In the current implementation, MLflow is mainly used for versioned reference and retrieval. It does not act as a full experiment tracking system for agent runs, model training, or execution metrics. Moreover, rollback is not implemented as an automatic platform feature. A rollback can be performed manually by changing the referenced prompt or schema version and redeploying the corresponding agent.

3.7.3 Provider and Model Configuration

VIP separates provider and model choices from the reusable agent runtime logic. When an agent is created, the selected provider, model, temperature, token limit, prompt URI, schema URIs, tools, and memory settings are stored as part of the agent configuration.

This configuration is stored in several forms. It is included in the frontend configuration, translated into the builder request, written into the generated `agent_config.yaml`, and persisted in PostgreSQL through configuration snapshots. At runtime, the generated agent reads this configuration and initializes the selected model provider accordingly.

In the current implementation, provider management is not fully dynamic from the user interface. The supported provider catalog is defined in code, and adding a new provider still requires code or configuration changes, as well as the corresponding environment variables for credentials and base URLs. Nevertheless, this design allows different agents to use different supported providers or models depending on quality, latency, and cost constraints.

The SLM-Evals project supports this provider abstraction by providing an evaluation pipeline for comparing models according to quality, latency, cost, and schema-compliance criteria. Its implementation and results are presented in Chapter 4.

3.7.4 Traceability and Current Persistence Limits

The storage architecture of VIP provides traceability for created agents and workflows. For agents, the platform stores identity metadata, selected tools, configuration snapshots, builder request snapshots, prompt and schema references, provider and model settings, runtime status, deployment metadata, and timestamps. For workflows, the platform stores the editable graph representation, the compiled runtime specification, validation information, status, version, and timestamps.

3.8 Deployment Architecture

The deployment architecture of VIP describes how generated agents are packaged, executed, and connected at runtime. While the previous sections focused on the logical architecture of the Agent Factory, the orchestrator, and the governance layer, this section focuses on the physical execution environment.

In VIP, generated agents are deployed as independent Docker containers. Each container exposes a FastAPI runtime service and joins a shared Docker network so that the orchestrator can invoke agents through internal HTTP endpoints. This deployment model supports service isolation, independent runtime execution, and clearer separation between build-time generation and runtime orchestration.

3.8.1 Dockerized Agent Services

Each generated agent in VIP is deployed as an independent Dockerized service. After the Agent Factory prepares the generated runtime structure, Docker builds an image containing the reusable agent runtime, the agent configuration, selected tools, dependency files, and the FastAPI entrypoint.

At runtime, each generated agent runs in its own container and exposes a standard HTTP interface. The main execution endpoint is the `/run` endpoint, which allows the orchestrator to invoke agents in a consistent way regardless of their internal role or configuration.

This deployment model reinforces the modular architecture of VIP. An extractor, mapper, generator, or sanitizer can run as a separate service while still following the same runtime contract. As a result, agents can be started, stopped, monitored, or replaced independently without modifying the rest of the platform.

Inside the container, generated agents listen on the same internal port. When needed, different host ports can be mapped to these internal ports for manual testing or debugging. However, service-to-service communication mainly relies on the internal Docker network rather than host-published ports.

3.8.2 Shared Docker Network and Service-to-Service Communication

Figure 33 presents a conceptual example of how Docker bridge networking enables communication between containers. In VIP, the same principle is applied through the external shared bridge network `vif_shared_net`.

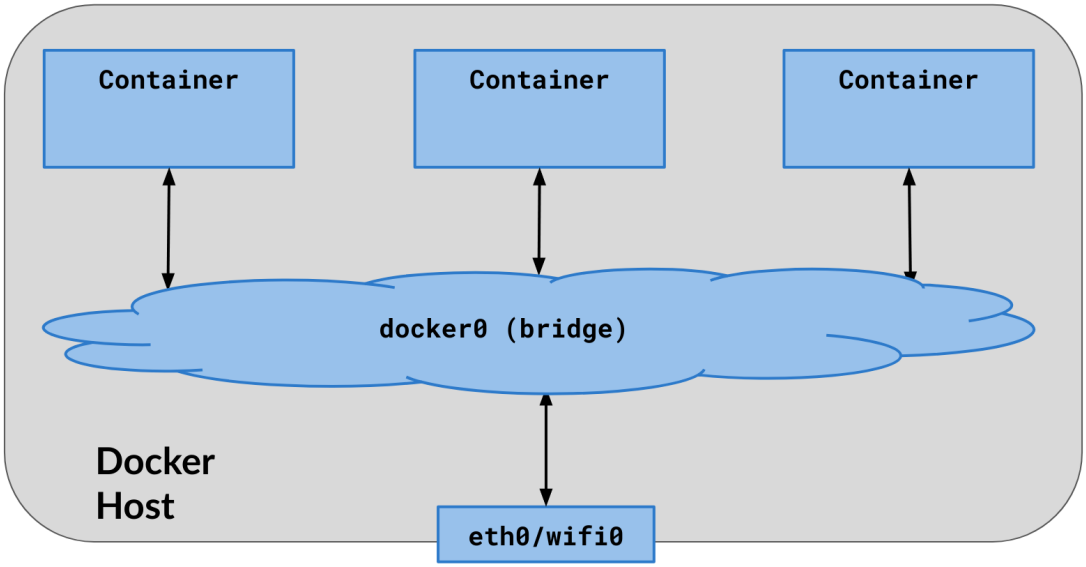

Figure 33: Example of container communication through a Docker bridge network

In the VIP deployment, the bridge network is used to support internal service-to-service communication. The orchestrator does not need to call agents through host addresses. Instead, it can reach each generated agent through its container name or network alias, using an internal endpoint such as `http://<container_name>:8000/run`. Host-published ports remain useful mainly for debugging or manual inspection.

All agent invocations are performed through REST-style HTTP calls. This keeps communication simple and technology-independent: as long as an agent exposes the expected runtime contract, it can be invoked by the orchestrator without embedding its internal logic into the orchestration layer.

3.8.3 Docker Image Optimization Strategy

Because each generated agent is deployed as a separate container, image size and runtime cleanliness are important deployment concerns. VIP addresses this by combining selective dependency resolution with a multi-stage Docker build strategy.

A multi-stage Docker build is a container image construction strategy where the build process is divided into separate stages. The first stage is used to install build dependencies and prepare the application artifacts, while the final stage keeps only the runtime elements required to execute the application. This avoids carrying unnecessary build tools, temporary files, and intermediate artifacts inside the final image.

Figure 34 illustrates the difference between a traditional single-stage Docker build and a multi-stage build. In a single-stage build, build dependencies and runtime dependencies may

remain together in the final image. In a multi-stage build, only the required runtime components are copied into the final image, which results in a smaller and cleaner container.

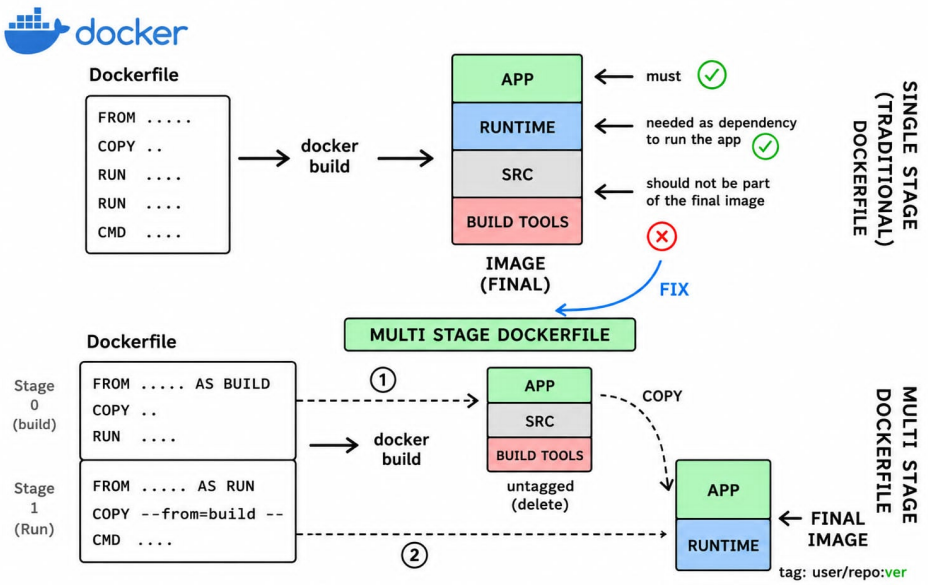

Figure 34: Single-stage build versus multi-stage Docker build

In VIP, this strategy is applied to each generated agent container. The build stage prepares the Python dependencies and creates the required wheels, while the runtime stage installs only the runtime packages and copies the generated agent files. As a result, the final agent image contains the FastAPI runtime, the agent configuration, the selected tools, and the required dependencies, without keeping unnecessary build-time packages.

This optimization is combined with the dependency resolution strategy introduced in the Agent Factory architecture. Instead of copying a global environment into every generated agent, VIP generates a tailored `requirements.txt` and `apt-runtime-packages.txt` based on the selected tools and runtime needs of each agent. This reduces the final image size and keeps each generated container aligned with the capabilities required by that specific agent.

3.9 Observability and Monitoring Architecture

Observability is an important part of the VIP platform because generated agents and workflows are executed across multiple independent services. In this context, monitoring a single service is not sufficient. The platform must be able to follow a request across the backend, the orchestrator, the generated agents, and the tools used during execution.

The observability architecture of VIP is mainly based on Laminar. Laminar is used to collect and store traces, spans, latency information, token usage, model-related information, and error details from the runtime layer. Custom Python logs are also used for local debugging, while OpenTelemetry is used to propagate trace context between services.

3.9.1 Laminar-Based Observability Design

During the first phase of the project, TraceAI was evaluated as a possible observability solution. However, this integration is not active in the current runtime. The platform later moved toward Laminar as the main observability solution because it can be deployed locally as a self-hosted Docker stack and provides access to traces, spans, metrics, execution runs, and a dedicated dashboard.

In the current architecture, Laminar is integrated mainly at the runtime layer. Generated agents initialize Laminar when they start, and they create spans for request handling, agent execution, tool execution, and model calls. The orchestrator also initializes Laminar and creates spans around workflow runs, workflow steps, and calls to generated agent endpoints.

The frontend backend does not emit Laminar spans directly. Instead, it queries Laminar data and exposes it to the frontend observability dashboard. This separation makes Laminar the main observability backend, while the frontend acts mainly as a visualization layer.

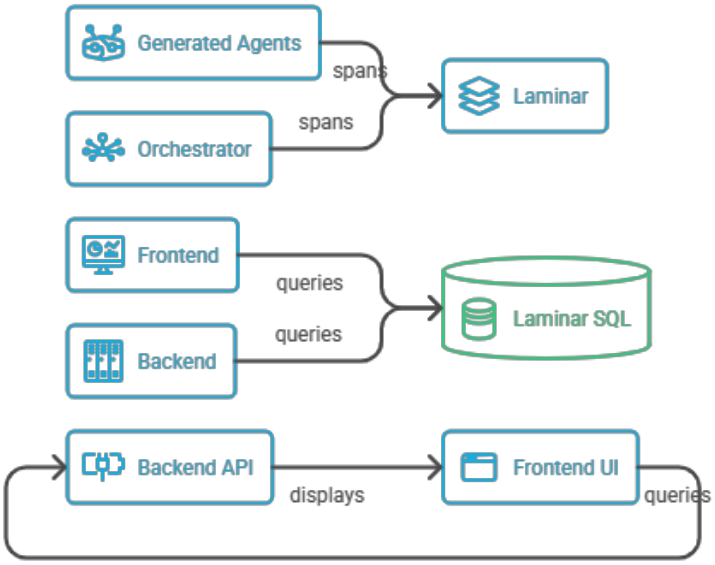

Figure 35: Laminar observability data flow in VIP

3.9.2 Execution Tracing Across Workflows

VIP uses a logical correlation identifier, called `trace_id`, to connect the different parts of a workflow execution. When a workflow is launched from the user interface, a trace identifier is created and sent with the execution request. If no identifier is provided, the backend can generate one before forwarding the request to the runtime orchestrator.

The orchestrator receives this `trace_id` and propagates it across the workflow execution.

For each workflow step, the same identifier is passed to the target generated agent through the agent runtime request. The generated agent then records this identifier as part of its Laminar span attributes, allowing orchestrator spans, agent spans, tool spans, and execution responses to be correlated.

This tracing mechanism is important because it allows the platform to understand where a workflow execution spends time, which agent was invoked, whether a step failed, and how the final response was produced.

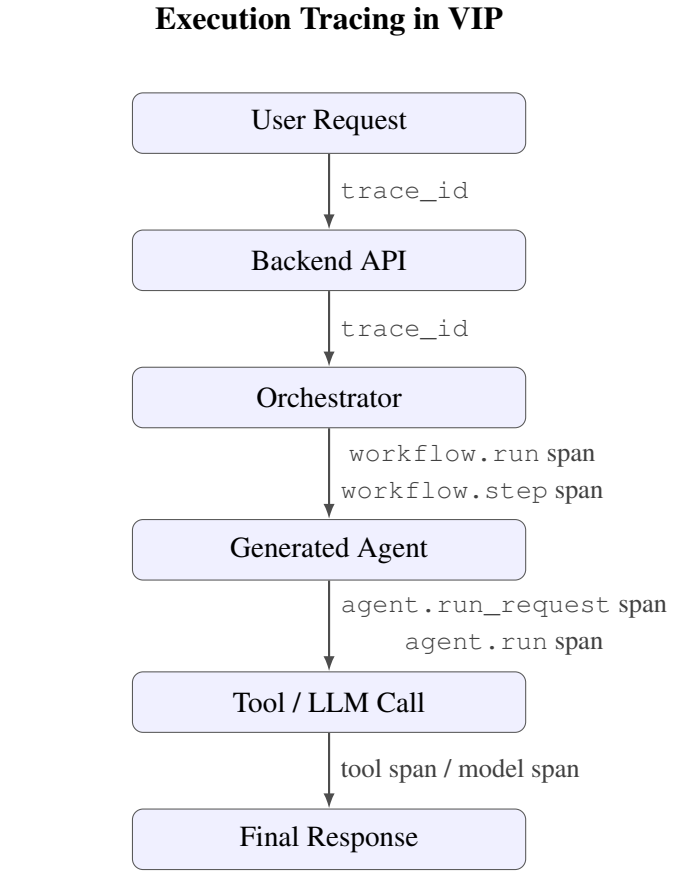

Figure 36: Execution tracing across VIP workflows

3.9.3 Token, Cost, and Latency Monitoring

VIP monitors several execution metrics that are important for operating agentic AI systems. Latency helps evaluate the responsiveness of generated agents and workflows. Token usage helps estimate model consumption and compare different executions. Cost monitoring supports financial control, especially when several agents and providers are used inside the same platform.

In the current implementation, generated agents directly produce token usage and latency metrics. Each agent measures the execution time of the request and extracts token usage from the model response when this information is available. These metrics are returned in the agent runtime response and aggregated by the orchestrator at workflow level.

Cost is handled differently. The generated agent does not manually calculate cost in its own response. Instead, cost information is obtained from Laminar trace data when the model instrumentation is able to capture it. As a result, token usage and latency are produced by the runtime execution path, while cost depends on the observability data available in Laminar.

These metrics are displayed in the frontend observability dashboard through backend endpoints that query Laminar. The dashboard can show information such as total runs, successful and failed runs, average latency, error rate, token usage, estimated cost, recent traces, and trace spans.

3.9.4 Logs and Runtime Monitoring

In addition to Laminar traces, VIP uses custom Python logs for local debugging and runtime inspection. Generated agents write logs inside their containers, typically under a local log file such as `logs/app.log`. The orchestrator also writes its own logs, for example under `logs/orchestrator.log`.

The same logs are also available through Docker stdout because the runtime loggers use console handlers. This means that developers can inspect runtime behavior either by reading log files inside the containers or by using Docker logging commands.

These logs complement Laminar but are not equivalent to structured traces. Laminar stores structured spans, attributes, outputs, status, latency, errors, and model-related information, while Python and Docker logs remain useful for development debugging.

The observability and monitoring architecture improves the operational control of VIP by making agent and workflow executions traceable across services. Laminar provides structured traces and metrics, custom logs support local debugging, and trace identifiers allow executions to be correlated across the backend, orchestrator, generated agents, tools, and model calls.

3.10 Chapter Summary

This chapter presented the system design and architecture of the VIP platform. It first introduced the architectural vision of VIP as a unified agentic hub designed to manage agents, workflows, providers, schemas, deployment, and observability in a controlled way.

The chapter then described the global architecture through the system context view, container view, and main platform components. These views showed how the frontend, backend, Agent Factory, generated agents, orchestrator, PostgreSQL, MLflow, Laminar, Docker environment, and LLM/SLM providers interact to support the lifecycle of configurable agents.

The agent lifecycle and Agent Factory architecture were then detailed. In VIP, agents are created from declarative configurations defining the provider, model, prompts, tools, and input/output schemas. The Agent Factory transforms these configurations into runnable agent services by assembling the reusable Agent Core, selecting the required tools, resolving dependencies, and preparing the generated runtime structure.

The chapter also explained the orchestration architecture, where generated agents can be executed individually or combined into multi-step workflows. Communication between agents is controlled through schema-based validation, and the adapter mechanism is used when payloads need to be transformed between workflow steps.

The data and governance layer clarified the roles of PostgreSQL, MLflow, and provider configuration. PostgreSQL stores platform metadata and runtime information, while MLflow manages prompt and schema references. The SLM-Evals project complements this architecture by supporting model and provider selection according to quality, latency, cost, and schema-compliance criteria.

The deployment architecture presented the physical execution model of VIP, where generated agents run as independent Dockerized FastAPI services connected through a shared Docker bridge network. It also introduced the use of multi-stage Docker builds and tailored dependency files to reduce final image size.

Finally, the observability architecture showed how Laminar, trace identifiers, token usage, latency metrics, cost information, and runtime logs support operational monitoring across the orchestrator, generated agents, tools, and model calls.

Overall, this chapter defined VIP as a modular, configurable, containerized, and observable agentic AI platform. The next chapter focuses on the implementation details of the main components, including the Agent Factory, generated agent runtime, workflow execution, observability integration, and the SLM-Evals evaluation pipeline.

Chapter 4

Implementation, Evaluation, and Results

4.1 Introduction

The previous chapter presented the system design and architecture of the VIP platform. This chapter moves from architectural design to practical implementation, evaluation, and results.

It first describes the implementation of the VIP platform, including the Agent Factory, generated agent runtime, workflow orchestration, Docker-based deployment, and observability integration. It then presents the SLM-Evals project, which was developed to compare LLM and SLM providers according to quality, latency, cost, schema compliance, stability, and domain relevance.

Finally, this chapter presents the evaluation protocol and discusses the obtained results. The objective is to assess whether the implemented solution satisfies the functional and non-functional objectives defined earlier in the report.

4.2 Implementation Environment

This section presents the technical environment used to implement the VIP platform. The objective is to give a simplified overview of the main technologies used for the frontend, backend services, metadata storage, prompt and schema management, deployment, and observability.

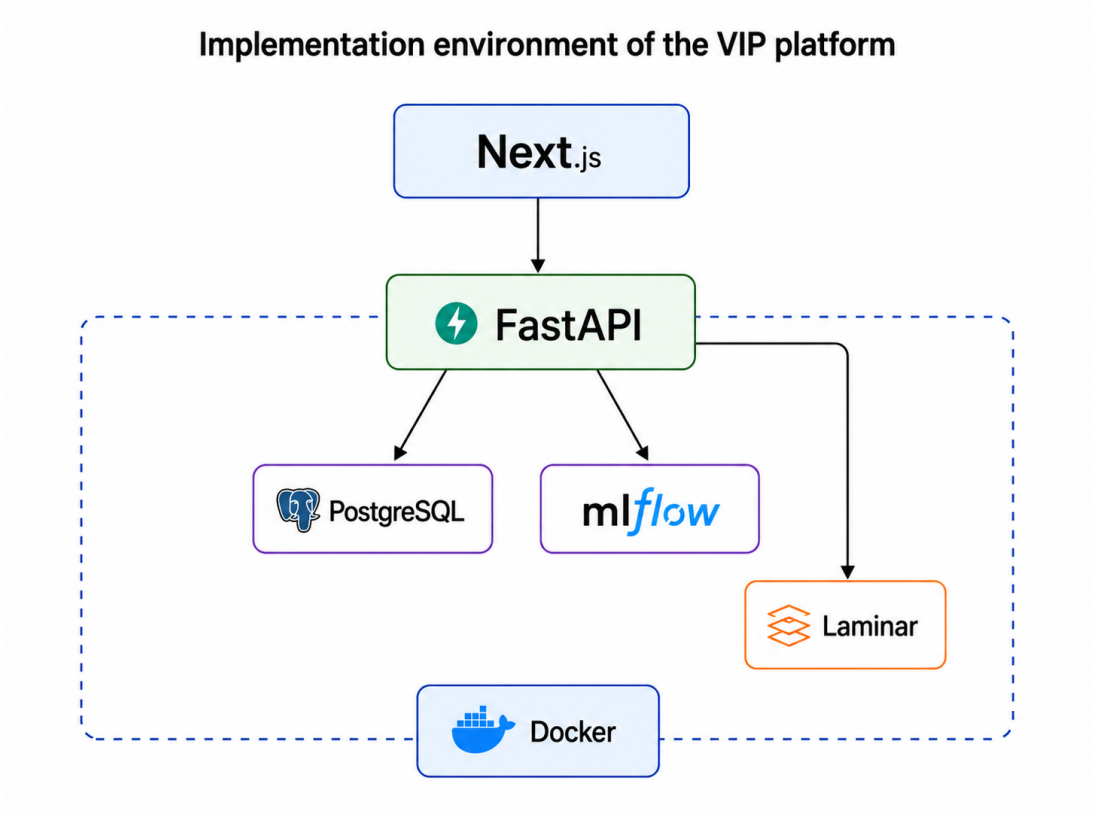

Figure 37: VIP implementation environment

This figure summarizes the main implementation stack of VIP. The frontend was developed with Next.js, while FastAPI was used for backend services, orchestration endpoints, and generated agent runtimes. PostgreSQL stores platform metadata such as agents, workflows, configuration snapshots, and runtime information. MLflow manages prompt and schema references, Docker supports containerized execution, and Laminar provides observability through traces, spans, latency, token usage, and model-call monitoring. The platform was developed and tested locally before being executed through Docker containers. This environment made it possible to validate agent generation, container execution, workflow orchestration, service-to-service communication, and observability integration. The following sections describe how these components were implemented, starting with the VIP platform and then moving to the SLM-Evals evaluation pipeline.

4.3 VIP Platform Implementation

4.3.1 Agent Factory Implementation

4.3.2 Generated Agent Runtime Implementation

4.3.3 Workflow Orchestration Implementation

4.3.4 Docker-Based Deployment Implementation

4.3.5 Observability Implementation

4.4 SLM-Evals Implementation

4.4.1 Objective of the Evaluation Pipeline

4.4.2 Evaluation Dataset and Test Cases

4.4.3 Providers and Models Evaluated

4.4.4 Prompting and Execution Protocol

4.4.5 Evaluation Metrics

4.4.6 Result Storage and Visualization

4.5 Evaluation Protocol

4.5.1 VIP Platform Validation Criteria

4.5.2 SLM-Evals Validation Criteria

4.6 Results and Discussion

4.6.1 VIP Platform Results

4.6.2 SLM-Evals Results

4.6.3 Discussion of Results

4.7 Chapter Summary